

Bioinformatics for Biotechnology Students

LUKAS KÄLL & ANDERS ANDERSSON

August 6, 2026

Contents

0.1	Sequence alignment	1
0.1.1	Introduction to Alignments	1
0.1.2	Algorithms for Pairwise Alignments	7
0.1.3	Protein Sequence Alignment	25
0.1.4	Multiple Sequence Alignments	34
0.1.5	Sequence Retrieval	53
0.2	Phylogenetics	61
0.2.1	Introduction to Phylogenetics	61
0.2.2	Distance-Based Phylogenetics	65
0.2.3	Character-Based Phylogenetics	73
0.2.4	Interpreting Phylogenetic Trees	77

0.1 Sequence alignment

0.1.1 Introduction to Alignments

Introduction to Sequence Alignments

A **sequence alignment** arranges two or more DNA, RNA, or protein sequences so that corresponding positions appear in the same column, making regions of similarity and difference immediately visible. Alignments are the central operation of bioinformatics: they underpin gene annotation, protein function prediction, evolutionary analysis, and sequence database search.

Illustration of a Sequence Alignment

GATTA -
GCT -AC

Each column represents one position in the alignment. Where both rows carry the same character the column is a **match** (G–G at position 1); different characters is a **mismatch** (A–C at position 2); a dash paired with a character is a **gap**, representing an insertion or deletion (indel) relative to the other sequence.

What is needed to systematically construct a pairwise alignment? To systematically construct a pairwise alignment we need three things.

- A scoring function, i.e. how is each position in the alignment scored.
- An alignment type, i.e. what type of alignment is to be constructed.
- An alignment algorithm, i.e. how do we optimise the score of an alignment to form the desired alignment type

Scoring function A scoring function, $d(x, y)$, giving the score of a column of any characters x and y . The characters could be DNA or RNA bases or amino acids, for now, let's think about them as DNA bases. A typical scoring function could be:

$$d(x, y) = \begin{cases} p & \text{if } x = y \\ g & \text{if } x = - \text{ or } y = - \\ n & \text{otherwise} \end{cases} \quad (1)$$

Here, p , is called a match score, n , a mismatch score, and g a gap penalty.

We score an alignment by summing up the scoring function over each position in an alignment. This allows us to evaluate if one alignment is better than another.

Example

Consider the alignment,

ACT
A -T

, using a match score of 2 and a gap penalty of -1. The scoring of this alignment is as follows:

- The first position has A aligned with A, which is a match. Assuming a match score of 2, this position contributes +2 to the total score.
- The second position has C aligned with a gap (-). With a gap penalty of -1, this position subtracts 1 from the total score.
- The third position has T aligned with T, another match. This contributes +2 to the total score.

Thus, the total score of the alignment ACT with A-T is calculated as, $+2 -1 +2 = 3$

Exercises

Alignment Type The alignment type specifies which part of each sequence must be covered by the optimal alignment.

Global alignment spans the full length of both sequences and penalises any unaligned ends. Use it when comparing closely related sequences of similar length — for example, orthologous genes from two species. The classic algorithm is [Needleman–Wunsch](#).

Local alignment finds the highest-scoring contiguous region of similarity anywhere within the two sequences, ignoring flanking regions entirely. Use it when only part of each sequence is expected to match — for example, detecting a conserved domain in otherwise divergent proteins. The classic algorithm is [Smith–Waterman](#).

Semi-global alignment requires one sequence to be fully covered while allowing unpenalised overhangs at either end of the other. Use it when a shorter sequence (e.g. a gene) should align entirely within a longer one (e.g. a genome). See [variants of Needleman–Wunsch](#).

Alignment Algorithm Given the scoring function and alignment type, we have a definition of what we want to achieve, i.e., there is a definition of optimality. Now we come to the question of *how* we obtain such optimality.

Exhaustive searches The simplest imaginable strategy is to enumerate every possible alignment of two sequences, score each one, and return the best. This is called an **exhaustive search**. For short sequences it works: you can write out all alignments by hand. At each position in the alignment, the algorithm must decide whether to match/mismatch the next character from each sequence, insert a gap in sequence 1, or insert a gap in sequence 2 — three choices that can be made independently at every step. The total number of alignments therefore grows exponentially with sequence length.

To make this concrete, consider two sequences of length n . A rough lower bound on the number of distinct global alignments is $\binom{2n}{n}$ — the number of ways to interleave n characters from each sequence before any gaps are added. For $n = 10$ this is already $\binom{20}{10} = 184,756$; for $n = 100$ it exceeds 10^{58} . Biological sequences are routinely hundreds to thousands of residues long, placing an exhaustive search entirely beyond computational reach.

The exhaustive approach fails not because any individual alignment is hard to score, but because the *number of candidates* grows faster than any practical computer can handle. This is the fundamental problem that dynamic programming solves.

Dynamic programming Dynamic programming avoids the combinatorial explosion by recognising that the optimal alignment of two full sequences must contain within it the optimal alignment of every pair of their prefixes — a property called **optimal substructure**. Each sub-problem therefore only needs to be solved once.

In practice this means:

- **Fill a matrix.** Cell (i, j) stores the best alignment score for the first i characters of sequence 1 against the first j characters of sequence 2. Each cell depends only on the three cells directly above, to the left, and diagonally above-left, so the whole table is filled in $O(mn)$ time.
- **Traceback.** Follow the recorded choices back from the end cell (global) or the highest-scoring cell (local) to read off the alignment.

This reduces an exponentially large search to a calculation that scales with the product of the sequence lengths, and it is the principle behind every alignment algorithm in the following chapters.

In the next chapters we will describe a set of such dynamic programming algorithms in detail, here is table summarizing them.

Sequence alignment types

Alignment Type	Definition	Typical Use Case	Algorithm
Global	Aligns full length of both sequences	Closely related sequences of similar length	Needleman–Wunsch
Local	Aligns subsequences with highest similarity	Motif/domain detection, different-length sequences	Smith–Waterman
Semi-global	Aligns one sequence entirely, allows overhangs in the other	Gene-to-genome, substring matching	Variants of Needleman–Wunsch

Representations of pairwise alignments

Before delving into the intricacies of the alignment algorithms, it's crucial to understand the matrix representation of alignments. This approach is not just a computational convenience but a fundamental framework that encapsulates the complexity of aligning biological sequences. Here's an overview of why matrices are used in sequence alignment and the significance of the traceback process.

Understanding Matrix Representation in Sequence Alignments

The Essence of Matrices in Alignments Aligning sequences involves comparing each character of one sequence with every character of another to identify regions of similarity or difference. Given the combinatorial explosion of possible alignments with even moderate-length sequences, a brute-force approach to comparison is computationally infeasible. This is where matrices come into play, offering a structured and efficient way to represent all possible alignments between two sequences.

How Matrices Facilitate Alignment A matrix for sequence alignment is essentially a grid. Each cell in this grid represents a comparison point between a character from one sequence and another from the second sequence. Each cell of the matrix corresponds to the characters in the two sequences being aligned. And a path from the top left cell to the bottom right cell of the matrix represents a global alignment.

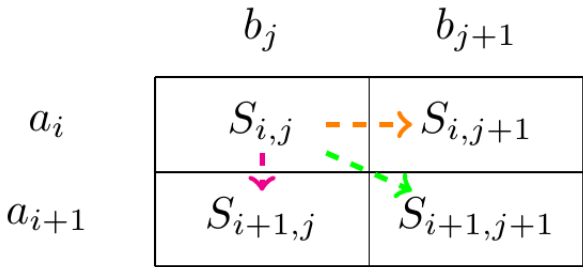


Figure 1: A step in the path representing an alignment.

If your alignment is in position a_i, b_j of two sequences $\mathbf{a} = (a_1, \dots, a_N)$ and $\mathbf{b} = (b_1, \dots, b_M)$ then the next step in the alignment is either a match/mismatch between a_{i+1}, b_{j+1} (diagonal movement), a delete $a_{i+1}, -$, (a vertical movement), or an insert $-, b_{j+1}$, (a horizontal movement).

Alignments as paths between the cells The path we follow tells us how the sequences should be aligned. A move diagonally signifies a match or mismatch, indicating that the characters from both sequences should be aligned with each other. Moving up or to the left indicates a gap in one of the sequences.

Examples of alignments An example of some matrix representations of an alignment is found in Figure 2.

The path of Alignment 1 is given in red, Alignment 2 in green, and Alignment 3 in blue. We also included a partial (local) alignment, Alignment 4 in orange.

Alignment 1 (red)	Alignment 2 (green)	Alignment 3 (blue)	Alignment 4 (orange)
--ACGTACT	ACGTAC-T	ACGTACT--	ACG
ACTACGT--	AC-TACGT	--ACTACGT	ACG

The matrix representation of alignments and the traceback process are not merely computational techniques; they are integral to understanding the relationships between biological sequences. By

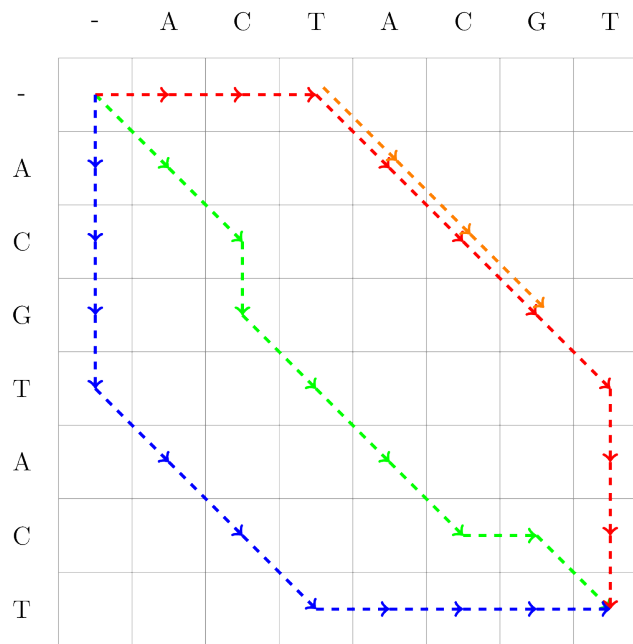


Figure 2: Matrix representation of three full length alignments of **ACGTACT** and **ACTACGT**, as well as one partial alignment of the same sequences.

breaking down the alignment process into a series of quantifiable steps, matrices allow us to systematically explore the vast space of alignment possibilities, leading us to the optimal alignment that best reflects the evolutionary or functional relationship between the sequences involved.

Exercises

Why Matrix Representations Are Useful Matrices are more than just a convenient way to visualize alignments — they form the mathematical foundation that allows us to apply efficient algorithms. The key property that makes them powerful is that they provide a **structured way to exploit the principle of optimal substructure**.

Optimal Substructure in Alignments When aligning two sequences, the score of the *best alignment* up to a given position (i, j) depends only on:

- the characters a_i and b_j , and
- the best scores of alignments leading into that position (from $(i-1, j-1)$, $(i-1, j)$, or $(i, j-1)$).

This means that once we have computed the optimal score up to (i, j) , we no longer need to remember *how* we reached that point. The matrix cell itself summarizes all relevant information about the past.

In other words, **any optimal scoring pathway of the subsequent part of the alignment is independent of the specific choices made earlier**. This independence allows us to build solutions incrementally, storing partial results in the matrix and avoiding redundant computations.

0.1.2 Algorithms for Pairwise Alignments

Needleman-Wunsch Algorithm: Global Alignment

Background Developed by Saul B. Needleman and Christian D. Wunsch in 1970 Needleman and Wunsch [1970], the Needleman-Wunsch algorithm was one of the first computational approaches to sequence alignment. Its introduction marked a significant advancement in bioinformatics, enabling the systematic and automated comparison of biological sequences.

Principle The algorithm is based on dynamic programming, a method that breaks down complex problems into simpler, smaller subproblems, solving each just once and storing their solutions. In the context of sequence alignment, it constructs an optimal global alignment by comparing every character of one sequence with every character of another, considering the costs of matches, mismatches, and gaps. The logic is that we build up an alignment one position at a time, using previously computed optimal alignments. We begin in the top left corner of the matrix representation of the alignment, where we still have not aligned any characters, and hence a score of zero. We then proceed to fill in the matrix, one cell at a time, until we reach the bottom right corner, which contains the score of the optimal alignment.

The definitions of the problem, and the solution Given two sequences $a_1, \dots, a_N$ and $b_1, \dots, b_M$, a scoring function $d(x, y)$, find a *global* alignment that gives an optimal (maximal) score.

The solution can be found by studying the dynamic programming matrix, S , of size $(N + 1, M + 1)$, by using the recursions defined in Equations (2) and (3).

$$S_{0,0} = 0, S_{i,0} = d(x, -) \cdot i \text{ for all } i, S_{0,j} = d(-, y) \cdot j \text{ for all } j \quad (2)$$

$$S_{i,j} = \max \begin{cases} S_{i-1,j-1} & +d(a_i, b_j) \\ S_{i-1,j} & +d(a_i, -) \\ S_{i,j-1} & +d(-, b_j). \end{cases} \quad (3)$$

We will walk through these steps more carefully below.

Process

1. **Initialization:** It starts by creating a scoring matrix where one sequence is aligned along the top and the other along the side. The first row and column are filled with gap penalties, increasing progressively to set up the basis for the algorithm according to Equation (2).
2. **Matrix Filling:** Each cell in the matrix is then filled based on the scores of adjacent cells (top, left, and diagonal), plus the score for matching or mismatching the corresponding characters, or introducing a gap. The choice of score at each cell reflects the highest score achievable from the possible alignments up to that point. Here we use the recursion given by Equation (3).
3. **Traceback:** Once the matrix is filled, the optimal alignment is determined by tracing back from the bottom-right corner to the top-left, following the path that resulted in the highest score. This path represents the optimal global alignment of the two sequences.
4. **Alignment Output:** The traceback path is used to construct the aligned sequences, introducing gaps as necessary, to maximize the alignment score based on the predefined scoring system. The final score of the alignment is given by the element, $S_{N,M}$.

Applications and Importance The Needleman-Wunsch algorithm is fundamental when the goal is to align entire sequences, providing a comprehensive view of their similarity. It's particularly valuable in evolutionary biology for comparing homologous sequences across different species, helping to infer phylogenetic relationships and evolutionary events. Moreover, it lays the foundation for understanding the principles of dynamic programming in bioinformatics, influencing the development of other alignment algorithms and tools.

Despite its computational intensity, especially for long sequences, the Needleman-Wunsch algorithm remains a crucial method for global sequence alignment, embodying the essence of comparing biological sequences in a mathematically rigorous and systematic way.

Examples

Example 1: Short sequences Here is an example alignment of the sequences **GAC** and **ACG** using Needleman-Wunsch, when we have a scoring function, $d(x, y) = \begin{cases} 1 & \text{if } x = y \\ -1 & \text{otherwise} \end{cases}$.

We start by filling in the borders of the matrix using the Equation (2), as shown in Figure 3.

	-	A	C	G
-	0	-1	-2	-3
G	-1			
A	-2			
C	-3			

Figure 3: Initialization of the gap penalty matrix for the Needleman-Wunsch algorithm, using Equation (2).

We then recursively fill in the other elements of the matrix in a row wise manner using Equation (3), as shown in Paragraph.

Filling in the NW matrix

We fill in the elements recursively, in a row-wise manner. Each cell's value is evaluated using Equation (3). The values of each recursion is spelled out under each image. We store trackers of which step we used to reach a certain cell, indicated by red arrows. Note that for some cells there are multiple optimal steps, i.e. paths that have the same score.

Given the filled in matrix, we can now track the optimal path from the bottom right element of the matrix, following the arrows back to the top-left element, as shown in Figure 4.

Example 2: Longer sequences Here we align the sequences $a = \text{TGCATTA}$ $b = \text{GCATTAC}$ when $d(x, y) = \begin{cases} 3 & \text{if } x = y \\ -2 & \text{if } x = - \text{ or } y = - \\ -1 & \text{otherwise} \end{cases}$.

The resulting matrix is found in Figure 5.

Exercise

Big-O Notation Big-O notation is used in computational science for describing how the running time or memory usage of an algorithm scales with a given factor. E.g. if we expect the running time

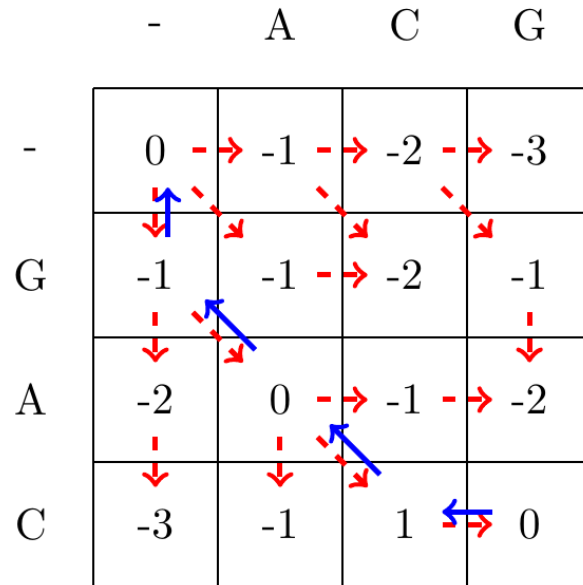


Figure 4: We trace the alignment backwards from the bottom-right corner to the top-left corner of the matrix, and mark the found optimal path with blue arrows.

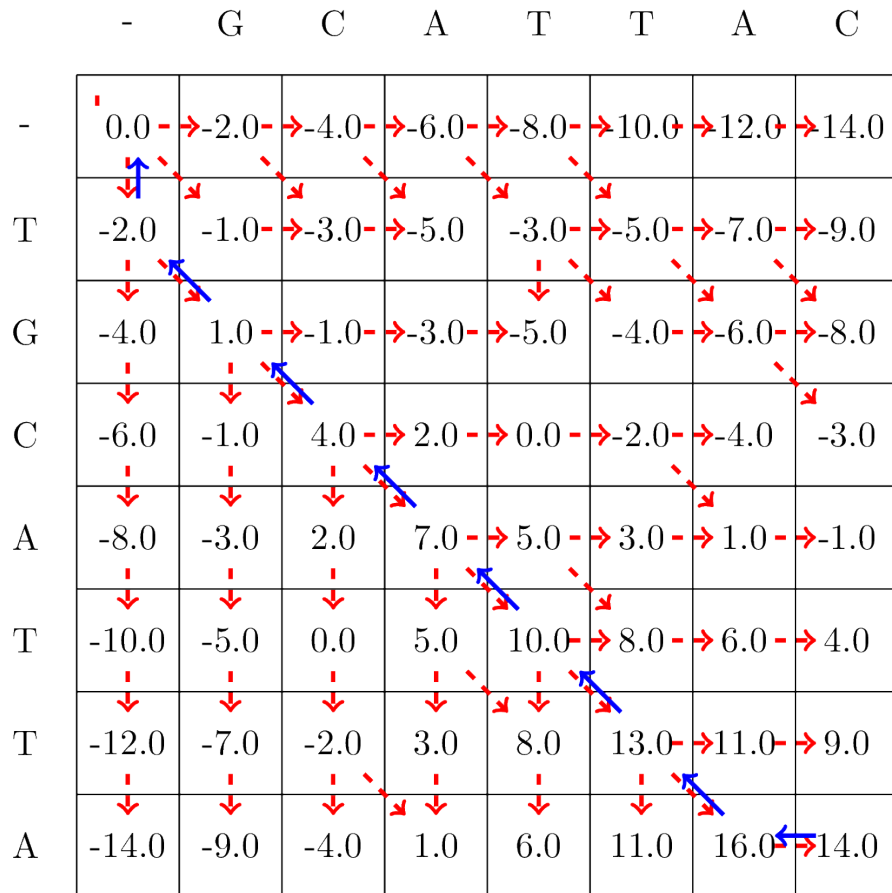


Figure 5: We follow the alignment backwards from the bottom-right corner to the top-left corner of the matrix, and mark the found optimal path with blue arrows.

to scale as $g(x)$ we write that the algorithm has complexity $\mathcal{O}(g(x))$. A more formal definition can be found at wikipedia.

In the case of Needleman-Wunsch we see that the number of calculations needed are proportional to the size of the dynamic programming matrix, which equals the product of the lengths of the sequences, $M \times N$. This results in a time complexity of $\mathcal{O}(MN)$, indicating that the time to complete the task scales proportionally with the product of the lengths of the two sequences.

In the same way memory usage also scales with $\mathcal{O}(MN)$, as the scoring matrix used to store intermediate results requires memory proportional to its size.

Big-O notation serves as a quick and effective tool for comparing different algorithms. For example, it allows us to see at a glance how the Needleman-Wunsch algorithm compares to other sequence alignment algorithms in terms of efficiency.

A useful comparison is the complexity of our initial proposition, to enumerate and calculate the scores for all possible alignments of two sequences. This can be done by calculating the number of alignments with k matches/mis-matches between the two sequences which is $\binom{M}{k}\binom{N}{k}$. If we assume that $N > M$ and sum this for all possible values of k , we get $\sum_{k=0}^M \binom{M}{k}\binom{N}{k} = \sum_{k=0}^M \binom{M}{M-k}\binom{N}{k} = \binom{N+M}{M} = \frac{(M+N)!}{M!N!}$ number of different alignments. This can be shown to follow $\mathcal{O}((\frac{e(N+M)}{M})^M)$ Lange [2002], Eddy [b].

Code: Global pairwise alignments

We will now make use of our definitions of a Needleman-Wunsch alignment to see how the algorithm transforms to actual code. The sections below will walk you through how this is done.

Scoring system for DNA sequences We setup the scoring system we need for the alignment of DNA sequences. Here we use a score system where gaps score -2 and miss matches are scored -1 and matches get a score of 3. You should absolutely try to change the scoring function at some point to see how the values affect the alignments.

```
# Returns the penalty for introducing a gap in the alignment.
def gap_penalty():
    return -2.0

# Returns the score for aligning two letters (could be nucleotides or amino acids).
# Matches get a positive score, mismatches a negative score, and gaps use the gap penalty.
def match_score(letterA, letterB):
    if letterA == '-' or letterB == '-':
        return gap_penalty() # Gap penalty if either is a gap
    elif letterA == letterB:
        return 3.0 # Match score
    else:
        return -1.0 # Mismatch penalty
```

Global alignments by Needleman-Wunsch Here follows an implementation of the Needleman-Wunsch pairwise alignment method. We want to align two sequences $a = a_1, \dots, a_M$ and $b = b_1, \dots, b_N$.

As said in last chapter, the dynamic programming matrix S is initiated as: $S_{i0} = g \cdot i, \forall i, S_{0j} = g \cdot j, \forall j$

This translates easily into two for loops filling in the upper row and the leftmost column.

The trace matrix: keeping track of the optimal path To reconstruct the optimal alignment after filling the score matrix S , we need to know which move (match/mismatch, insertion, or deletion) led to each cell's score. For this, we use a trace matrix. Each element in the trace matrix contains a pair of values (di, dj) indicating the direction of the optimal step that led to the current cell:

- $(-1, -1)$: Diagonal move (match or mismatch, aligning a_i with b_j)
- $(-1, 0)$: Upwards move (gap in sequence B, aligning a_i with a gap)
- $(0, -1)$: Leftwards move (gap in sequence A, aligning b_j with a gap)

By following these pointers from the bottom-right cell back to the top-left, we can reconstruct the optimal alignment by inserting gaps where needed. This process is called traceback.

Here we initiate pointers in the form of a trace matrix. Each element in the trace matrix contains the difference in index between the current cell and the optimal step that lead to the current cell.

```
# Initialize the dynamic programming matrices for global alignment.
# S: score matrix, trace: matrix to keep track of the optimal path.
def initiate_global_dp(m,n):
    S = np.zeros((m,n)) # Score matrix of size m x n, initialized to 0
    trace = np.zeros((m,n,2)) # Trace matrix to store direction of optimal move
    S[0,0] = 0.
    trace[0,0,:] = (0.,0.)
    # Initialize first column: gap penalties for seqA
    for i in range(1,m):
        S[i,0] = i * gap_penalty()
        trace[i,0,:] = (-1,0) # Came from above (gap in seqB)
```

```
# Initialize first row: gap penalties for seqB
for j in range(1,n):
    S[0,j] = j * gap_penalty()
    trace[0,j,:] = (0, -1) # Came from left (gap in seqA)
return S,trace
```

Subsequently, the rest of S is filled as: $S_{ij} = \max \begin{cases} S_{i-1,j-1} & +d(a_i, b_j) \\ S_{i-1,j} & +d(a_i, -) \\ S_{i,j-1} & +d(-, b_j) \end{cases}$

This recursion is easily transformed into for-loops. We are free to select the order the matrix is filled in so we select to fill in row per row, i.e. rows become the inner loop, the columns the outer loop.

Again we keep track of the move that lead to a certain position by filling in the `trace` matrix.

```
# Perform global alignment using the Needleman -Wunsch algorithm.
# Returns the aligned sequences (with gaps) and optionally prints the score matrix.
def global_align(seqA,seqB,print_dynamic_matrix = False):
    # Add 1 to lengths for DP matrix (to account for initial gap row/column)
    m, n = len(seqA)+1, len(seqB)+1
    S,trace = initiate_global_dp(m,n)
    # Fill in the rest of the dynamic programming matrix
    for i in range(1,m):
        for j in range(1,n):
            # Calculate scores for match/mismatch, deletion, and insertion
            # Note: i -1 and j -1 because Python uses 0 -based indexing
            match = S[i -1,j -1] + match_score(seqA[i -1],seqB[j -1])
            delete = S[i -1,j] + match_score(seqA[i -1],'-')
            insert = S[i,j -1] + match_score('-',seqB[j -1])
            # Choose the maximum score and record the move in trace
            S[i,j] = max(match,delete,insert)
            if match >= max(insert,delete):
                trace[i,j,:] = (-1, -1) # Diagonal move: match/mismatch
            elif delete >= insert:
                trace[i,j,:] = (-1,0) # Up: gap in seqB (deletion)
            else:
                trace[i,j,:] = (0, -1) # Left: gap in seqA (insertion)
    if print_dynamic_matrix:
        print_dynamic(seqA,seqB,S,trace) # Optional: print the score matrix
    return format_alignment(seqA,seqB,S,trace) # Traceback to get alignment
```

Now everything is set. We can try the code for any of our favorite sequences. One can toggle the printout of the dynamic programming matrix by a boolean flag as a third argument.

```
# Example: align two short DNA sequences and print the result
seqA,seqB = global_align("GATTA","GCTAC",True)
print_alignment(seqA,seqB)
```

I add a couple of extra alignments, check them manually as an exercise before the exam. Also try to run a couple of them yourself.

```
# Another example: align two longer sequences
seqA,seqB = global_align("TGCATTA","GCATTAC",True)
print_alignment(seqA,seqB)

# Try aligning even longer sequences for practice
seqA,seqB = global_align("CTATCTCGCTATCCA","CTACGCTATTTC",True)
print_alignment(seqA,seqB)
```

Smith-Waterman Algorithm: Local Alignment

Background Developed by Temple F. Smith and Michael S. Waterman in 1981, the Smith-Waterman algorithm is the classical method for forming local sequence alignments [1981]. It was designed to identify regions of similarity within long sequences of DNA, RNA, or proteins, allowing for variability in length and composition.

Principle The Smith-Waterman algorithm employs dynamic programming to construct an alignment matrix that scores local alignments between two sequences. It initializes the matrix and fills it based on match, mismatch, and gap penalties, iteratively computing the highest possible score for any contiguous alignment segment. This means that our alignment can start and end at any position in the sequences, focusing on the most similar regions rather than aligning the entire length of both sequences. This is obtained by resetting the score to zero if we get a negative score.

Implementation The implementation of the Smith-Waterman algorithm involves initializing an alignment matrix with zeros, and then iteratively updating its cells based on the scores of adjacent cells and the scoring schema (match, mismatch, and gap penalties). Key considerations include the choice of scoring parameters and the method for traceback to reconstruct the optimal local alignment.

Applications The Smith-Waterman algorithm is crucial for various applications in genomic research, drug discovery, and evolutionary studies. It is particularly useful in situations where the most informative element is a portion of the available data, such as identifying conserved motifs within a larger genomic sequence or aligning sequences that include regions of high variation.

The definitions of the problem, and the solution Given two sequences $a_1, \dots, a_N$ and $b_1, \dots, b_M$, a scoring function $d(x, y)$, find a *local* alignment that gives an optimal (maximal) score.

The solution can be found by studying the dynamic programming matrix, S , of size $(N + 1, M + 1)$, by using the recursions defined in equations (??) and (4).

$$\begin{aligned}
 S_{0,0} &= 0, \\
 S_{i,0} &= 0, & \text{for all } i \\
 S_{0,j} &= 0, & \text{for all } j \\
 S_{i,j} &= \max \begin{cases} 0 \\ S_{i-1,j-1} + d(a_i, b_j) \\ S_{i-1,j} + d(a_i, -) \\ S_{i,j-1} + d(-, b_j). \end{cases} \quad (4)
 \end{aligned}$$

Example Here is an example alignment of the sequences GAC and ACG using Smith-Waterman, when we have a scoring function, $d(x, y) = \begin{cases} 1 & \text{if } x = y \\ -1 & \text{otherwise} \end{cases}$.

We start by filling in the borders of the matrix using Equation (??), as shown in Figure 6.

We then recursively fill in the other elements of the matrix in a row wise manner using Equation (4), as shown in Figure 7.

Given the filled in matrix, we can now track the optimal path from the maximal element of the matrix, following the arrows back to the first element in the path with a score of zero, as shown in Figure 8.

Exercises

	-	A	C	G
-	0	0	0	0
G	0			
A	0			
C	0			

Figure 6: Initialization of the dynamic programming matrix, using Equation (??). For Smith-Waterman this equates to setting the elements of the first row and column to 0.

	-	A	C	G
-	0	0	0	0
G	0	0	0	1
A	0	1	0	0
C	0	0	2	1

Figure 7: Filling in the matrix. We fill in the elements recursively, in a row-wise manner. Each cell's value is evaluated using Equation (4). We store trackers of which step we used to reach a certain cell, indicated by red arrows. Note that for some cells there are multiple optimal steps, i.e. paths that have the same score.

	-	A	C	G
-	0	0	0	0
G	0	0	0	1
A	0	1	0	0
C	0	0	2	1

Figure 8: We follow the alignment backwards from the matrix element with the largest value, $\max_{i,j} S_{ij}$, to the first encountered cell with a value of zero where we stop, and mark the found optimal path with blue arrows.

Code: Local Pairwise alignment

Scoring system for DNA sequences We setup the scoring system we need for the alignment of DNA sequences. Here we use a score system where gaps score -2 and miss matches are scored -1 and matches get a score of 3.

```
def gap_penalty():
    return -2.0

def match_score(letterA,letterB):
    if letterA == '-' or letterB == '-':
        return gap_penalty()
    elif letterA == letterB:
        return 3.0
    else:
        return -1.0
```

Local alignments using Smith-Waterman Smith-Waterman alignments are similar to the ones of Needleman-Wunsch. The difference sits in the initiation of the dynamic programming matrix, and how we trace the most optimal alignment. We will implement these difference by redefining some functions.

First the initiation of the dynamic programming matrix S : $S_{i0} = 0, \forall i, S_{0j} = 0, \forall j$

```
# Initiating dynamic programming matrices, S and trace
def initiate_local_dp(m,n):
    S = np.zeros((m,n))
    trace = np.zeros((m,n,2))
    S[0,0] = 0.
    trace[0,0,:] = (0.,0.)
    for i in range(1,m):
        S[i,0] = 0
        trace[i,0,:] = (0,0)
    for j in range(1,n):
        S[0,j] = 0
        trace[0,j,:] = (0,0)
    return S,trace
```

Subsequently, the rest of S is filled as: $S_{ij} = \max \begin{cases} S_{i-1,j-1} & +d(a_i, b_j) \\ S_{i-1,j} & +d(a_i, -) \\ S_{i,j-1} & +d(-, b_j) \\ 0 \end{cases}$

```
def local_align(seqA,seqB,print_dynamic_matrix = False):
    # Initiating variables
    m, n = len(seqA)+1, len(seqB)+1
    S,trace = initiate_local_dp(m,n)
    # Fill in the rest of the dynamic programming matrix
    for i in range(1,m):
        for j in range(1,n):
            match = S[i-1][j-1] + match_score(seqA[i-1],seqB[j-1])
            delete = S[i-1,j] + match_score(seqA[i-1],'-')
            insert = S[i,j-1] + match_score('-',seqB[j-1])
            S[i,j] = max(match, delete, insert, 0.)
            if match >= max(delete,insert,0.):
                trace[i,j,:] = (-1, -1.)
```

```

        elif delete >= max(insert,0.):
            trace[i,j,:] = ( -1.,0.)
        elif insert >= 0.:
            trace[i,j,:] = (0., -1.)
        else:
            trace[i,j,:] = (0.,0.)
    if print_dynamic_matrix:
        print_dynamic(seqA,seqB,S)
    return format_local_alignment(seqA,seqB,S,trace)

```

We also need a slightly different method to trace the the alignment. It is not essential you understand this code.

```

seqA,seqB = local_align("GATTA","GCTAC",True)
print_alignment(seqA,seqB)

```

```

-   G   C   T   A   C
-   0.0 0.0 0.0 0.0 0.0 0.0 0.0
G   0.0 3.0 1.0 0.0 0.0 0.0 0.0
A   0.0 1.0 2.0 0.0 3.0 1.0
T   0.0 0.0 0.0 5.0 3.0 2.0
T   0.0 0.0 0.0 3.0 4.0 2.0
A   0.0 0.0 0.0 1.0 6.0 4.0

```

Best score: 6.0

```

GATTA
G -CTA

```

```

seqA,seqB = local_align("GCGATTA","GCTTAC",True)
print_alignment(seqA,seqB)

```

```

-   G   C   T   T   A   C
-   0.0 0.0 0.0 0.0 0.0 0.0 0.0
G   0.0 3.0 1.0 0.0 0.0 0.0 0.0
C   0.0 1.0 6.0 4.0 2.0 0.0 3.0
G   0.0 3.0 4.0 5.0 3.0 1.0 1.0
A   0.0 1.0 2.0 3.0 4.0 6.0 4.0
T   0.0 0.0 0.0 5.0 6.0 4.0 5.0
T   0.0 0.0 0.0 3.0 8.0 6.0 4.0
A   0.0 0.0 0.0 1.0 6.0 11.0 9.0

```

Best score: 11.0

```

GATTA
GCTTA

```

```

seqA,seqB = local_align("CTATCTCGCTATCCA","CTACGCTATTTC",True)
print_alignment(seqA,seqB)

```

```

-   C   T   A   C   G   C   T   A   T   T   T   C   A
-   0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
C   0.0 3.0 1.0 0.0 3.0 1.0 3.0 1.0 0.0 0.0 0.0 0.0 3.0 1.0
T   0.0 1.0 6.0 4.0 2.0 2.0 1.0 6.0 4.0 3.0 3.0 3.0 1.0 2.0
A   0.0 0.0 4.0 9.0 7.0 5.0 3.0 4.0 9.0 7.0 5.0 3.0 2.0 4.0
T   0.0 0.0 3.0 7.0 8.0 6.0 4.0 6.0 7.0 12.0 10.0 8.0 6.0 4.0
C   0.0 3.0 1.0 5.0 10.0 8.0 9.0 7.0 5.0 10.0 11.0 9.0 11.0 9.0

```

T	0.0	1.0	6.0	4.0	8.0	9.0	7.0	12.0	10.0	8.0	13.0	14.0	12.0	10.0
C	0.0	3.0	4.0	5.0	7.0	7.0	12.0	10.0	11.0	9.0	11.0	12.0	17.0	15.0
G	0.0	1.0	2.0	3.0	5.0	10.0	10.0	11.0	9.0	10.0	9.0	10.0	15.0	16.0
C	0.0	3.0	1.0	1.0	6.0	8.0	13.0	11.0	10.0	8.0	9.0	8.0	13.0	14.0
T	0.0	1.0	6.0	4.0	4.0	6.0	11.0	16.0	14.0	13.0	11.0	12.0	11.0	12.0
A	0.0	0.0	4.0	9.0	7.0	5.0	9.0	14.0	19.0	17.0	15.0	13.0	11.0	14.0
T	0.0	0.0	3.0	7.0	8.0	6.0	7.0	12.0	17.0	22.0	20.0	18.0	16.0	14.0
C	0.0	3.0	1.0	5.0	10.0	8.0	9.0	10.0	15.0	20.0	21.0	19.0	21.0	19.0
C	0.0	3.0	2.0	3.0	8.0	9.0	11.0	9.0	13.0	18.0	19.0	20.0	22.0	20.0
A	0.0	1.0	2.0	5.0	6.0	7.0	9.0	10.0	12.0	16.0	17.0	18.0	20.0	25.0

Best score: 25.0

CT -CGCTA -TCCA

CTACGCTATTTCA

Semi-global Alignment

Background We also have a third type of alignments, semi-global alignments, which are also known as semi-local or glocal alignments.

Principle Here we cross breed the Needleman-Wunsch and the Smith-Waterman algorithm, by initiating the dynamic programming matrix as for a local alignment, but using the same recursion as in the global alignment.

Implementation The implementation of the Semi-global alignment algorithm involves initializing an alignment matrix with zeros, and then iteratively updating its cells based on the scores of adjacent cells and the scoring schema (match, mismatch, and gap penalties).

Applications It is particularly useful in situations where we strive to align a short sequence to a longer sequence.

The definitions of the problem, and the solution Given two sequences $a_1, \dots, a_N$ and $b_1, \dots, b_M$, a scoring function $d(x, y)$, find a *semi-global* alignment that gives an optimal (maximal) score.

The solution can be found by studying the dynamic programming matrix, S , of size $(N + 1, M + 1)$, by using the recursions defined in equations (??) and (5).

$$\begin{aligned}
 S_{0,0} &= 0, \\
 S_{i,0} &= 0, & \text{for all } i \\
 S_{0,j} &= 0, & \text{for all } j \\
 S_{i,j} &= \max \begin{cases} S_{i-1,j-1} & +d(a_i, b_j) \\ S_{i-1,j} & +d(a_i, -) \\ S_{i,j-1} & +d(-, b_j). \end{cases} \quad (5)
 \end{aligned}$$

The optimal alignment is found by backtracing from the maximal bottommost or rightmost element, $\max(\max_i S_{i,M}, \max_j S_{N,j})$, to the first encountered leftmost or topmost (0) element.

Summary: Comparing Global, Local, and Semi-global Alignment Sequence alignment algorithms can be broadly categorized into three types: global (Needleman-Wunsch), local (Smith-Waterman), and semi-global alignment. Each serves a distinct purpose depending on the biological question and the nature of the sequences being compared.

- **Global alignment (Needleman-Wunsch)** aligns two sequences from end to end, optimizing the alignment across their entire lengths. It is best suited for sequences of similar length and overall similarity.
- **Local alignment (Smith-Waterman)** finds the highest-scoring matching region (subsequence) between two sequences, making it ideal for identifying conserved domains or motifs within otherwise divergent sequences.
- **Semi-global alignment** is a hybrid approach, allowing for gaps at the ends of one or both sequences without penalty. This is particularly useful when aligning a shorter sequence (such as a sequencing read or a gene) to a longer reference (such as a chromosome or genome).

The table below summarizes the key differences:

Alignment Type	Matrix Initialization	Recursion Used	Traceback Start	Traceback End	Typical Case	Use
Global	Gap penalties	Global (Needleman-Wunsch)	Bottom-right cell	Top-left cell	Full-length comparison of similar sequences	
Local	Zeros	Local (Smith-Waterman)	Max cell anywhere	First zero	Motif/domain search, conserved regions	
Semi-global	Zeros	Global (Needleman-Wunsch)	Max in last row/column	First zero in row/col	Short-to-long sequence alignment	

Understanding these differences helps in selecting the appropriate algorithm for a given biological problem.

Code: Semi-global pairwise alignment

Scoring system for DNA sequences We setup the scoring system we need for the alignment of DNA sequences. Here we use a score system where gaps score -2 and miss matches are scored -1 and matches get a score of 3.

```
def gap_penalty():
    return -2.0

def match_score(letterA,letterB):
    if letterA == '-' or letterB == '-':
        return gap_penalty()
    elif letterA == letterB:
        return 3.0
    else:
        return -1.0
```

Semi-global alignments Yet another type of alignments are the semi-global alignments. Here we will be able to reuse the code for the global score, but we will initiate the dynamic programming matrix as for the local alignment. We also need to redefine how to read the alignment.

First we initiate the dynamic programming matrix S : $S_{i0} = 0, \forall i$, $S_{0j} = 0, \forall j$

```
# Initiating dynamic programming matrices, S and trace
def initiate_semiglobal_dp(m,n):
    S = np.zeros((m,n))
    trace = np.zeros((m,n,2))
    S[0,0] = 0.
    trace[0,0,:] = (0.,0.)
    for i in range(1,m):
        S[i,0] = 0
        trace[i,0,:] = (-1,0)
    for j in range(1,n):
        S[0,j] = 0
        trace[0,j,:] = (0, -1)
    return S,trace

def format_semiglobal_alignment(seqA,seqB,S,trace):
    outA,outB = "",""
    m,n = S.shape[0] -1, S.shape[1] -1
    i,j = S[:,n].argmax(), S[m,:].argmax()
    if S[i,n] > S[m,j]:
        print("Best score: " + str(S[i,n]))
        j = n
        # point the alignment from (m,n) to here
        for ix in range(i+1,m+1):
            print(ix,n)
            trace[ix,n,:] = (-1,0)
    else:
        print("Best score: " + str(S[m,j]))
        i = m
        # point the alignment from (m,n) to here
        for ix in range(j+1,n+1):
            print(m,ix)
            trace[m,ix,:] = (0, -1)
    i,j = m,n
```

```

while min(trace[i,j])<0:
    di,dj = trace[i,j]
    i += int(di)
    j += int(dj)
    if di == 0:
        outA = " -" + outA
    else:
        outA = seqA[i] + outA
    if dj == 0:
        outB = " -" + outB
    else:
        outB = seqB[j] + outB
return outA,outB

```

Subsequently, the rest of S is filled as:

$$S_{ij} = \max \begin{cases} S_{i-1,j-1} + d(a_i, b_j) \\ S_{i-1,j} + d(a_i, -) \\ S_{i,j-1} + d(-, b_j) \end{cases}$$

```

def semiglobal_align(seqA,seqB,print_dynamic_matrix = False):
    # Initiating variables
    m, n = len(seqA)+1, len(seqB)+1
    S,trace = initiate_semiglobal_dp(m,n)
    # Fill in the rest of the dynamic programming matrix
    for i in range(1,m):
        for j in range(1,n):
            match = S[i -1][j -1] + match_score(seqA[i -1],seqB[j -1])
            delete = S[i -1,j] + match_score(seqA[i -1],' -')
            insert = S[i,j -1] + match_score(' -',seqB[j -1])
            S[i,j] = max(match, delete, insert)
            if match >= max(delete,insert):
                trace[i,j,:] = ( -1, -1.)
            elif delete >= insert:
                trace[i,j,:] = ( -1.,0.)
            else:
                trace[i,j,:] = (0., -1.)
    if print_dynamic_matrix:
        print_dynamic(seqA,seqB,S)
    return format_semiglobal_alignment(seqA,seqB,S,trace)

seqA,seqB = semiglobal_align("CTATCTATCTCGCTATCCAA","CTACGCTATTTCA",True)
print_alignment(seqA,seqB)

```

-	C	T	A	C	G	C	T	A	T	T	T	C	A	
-	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
C	0.0	3.0	1.0	-1.0	3.0	1.0	3.0	1.0	-1.0	-1.0	-1.0	-1.0	3.0	1.0
T	0.0	1.0	6.0	4.0	2.0	2.0	1.0	6.0	4.0	2.0	2.0	2.0	1.0	2.0
A	0.0	-1.0	4.0	9.0	7.0	5.0	3.0	4.0	9.0	7.0	5.0	3.0	1.0	4.0
T	0.0	-1.0	2.0	7.0	8.0	6.0	4.0	6.0	7.0	12.0	10.0	8.0	6.0	4.0
C	0.0	3.0	1.0	5.0	10.0	8.0	9.0	7.0	5.0	10.0	11.0	9.0	11.0	9.0
T	0.0	1.0	6.0	4.0	8.0	9.0	7.0	12.0	10.0	8.0	13.0	14.0	12.0	10.0
A	0.0	-1.0	4.0	9.0	7.0	7.0	8.0	10.0	15.0	13.0	11.0	12.0	13.0	15.0
T	0.0	-1.0	2.0	7.0	8.0	6.0	6.0	11.0	13.0	18.0	16.0	14.0	12.0	13.0
C	0.0	3.0	1.0	5.0	10.0	8.0	9.0	9.0	11.0	16.0	17.0	15.0	17.0	15.0
T	0.0	1.0	6.0	4.0	8.0	9.0	7.0	12.0	10.0	14.0	19.0	20.0	18.0	16.0

C	0.0	3.0	4.0	5.0	7.0	7.0	12.0	10.0	11.0	12.0	17.0	18.0	23.0	21.0
G	0.0	1.0	2.0	3.0	5.0	10.0	10.0	11.0	9.0	10.0	15.0	16.0	21.0	22.0
C	0.0	3.0	1.0	1.0	6.0	8.0	13.0	11.0	10.0	8.0	13.0	14.0	19.0	20.0
T	0.0	1.0	6.0	4.0	4.0	6.0	11.0	16.0	14.0	13.0	11.0	16.0	17.0	18.0
A	0.0	-1.0	4.0	9.0	7.0	5.0	9.0	14.0	19.0	17.0	15.0	14.0	15.0	20.0
T	0.0	-1.0	2.0	7.0	8.0	6.0	7.0	12.0	17.0	22.0	20.0	18.0	16.0	18.0
C	0.0	3.0	1.0	5.0	10.0	8.0	9.0	10.0	15.0	20.0	21.0	19.0	21.0	19.0
C	0.0	3.0	2.0	3.0	8.0	9.0	11.0	9.0	13.0	18.0	19.0	20.0	22.0	20.0
A	0.0	1.0	2.0	5.0	6.0	7.0	9.0	10.0	12.0	16.0	17.0	18.0	20.0	25.0
A	0.0	-1.0	0.0	5.0	4.0	5.0	7.0	8.0	13.0	14.0	15.0	16.0	18.0	23.0

Best score: 25.0

20 13

CTATCTATCT -CGCTA -TCCAA

- - - - - - -CTACGCTATTTC A -

0.1.3 Protein Sequence Alignment

Aligning Proteins

Much of the content of this chapter is inspired by Eddy [a], which can be read side by side with this chapter.

Why Align Protein Rather Than DNA Sequences? Aligning protein sequences rather than DNA sequences provides several advantages, especially when considering evolutionary and functional analyses:

Greater Conservation of Amino Acids Over Nucleotides Aligning protein sequences instead of DNA sequences is often more informative for several reasons:

- **Conservation of Amino Acids:** Amino acid sequences tend to be more conserved than their corresponding DNA sequences. This is largely due to synonymous mutations, which do not alter the amino acid sequence despite changes in the DNA sequence. Such mutations are more common than might be expected if only considering the mutation frequencies of non-synonymous changes. This higher conservation level makes protein alignments particularly useful in phylogenetic analysis and in identifying functionally important regions.
- **Conservative Nature of Non-synonymous Mutations:** Even when non-synonymous mutations occur, they frequently result in amino acids with similar properties, such as polarity or size. This conservation of biochemical properties is less random and more structured compared to what would be expected by random mutation frequencies alone. This implies that natural selection favors mutations that preserve functionality, further supporting the usefulness of protein sequence alignment in evolutionary studies.

Scoring Functions for Amino Acid Sequences

Need for Specific Scoring Systems While it is technically possible to use the same types of scoring functions for both DNA and protein sequences, protein alignments benefit from the use of specialized scoring matrices. Here's why:

- **Complexity of Protein Sequences:** Proteins are composed of 20 different amino acids, each with unique chemical properties, as opposed to the 4 nucleotide bases of DNA. This complexity means that the impact of substituting one amino acid for another can vary greatly

Alignment of Protein Sequences Using BLOSUM and PAM Scoring Matrices

Introduction to Scoring Matrices Protein sequence alignment is a fundamental task in bioinformatics, crucial for structure prediction, functional annotation, and evolutionary studies. Two of the most commonly used scoring matrices in protein alignments are BLOSUM (Blocks Substitution Matrix) and PAM (Point Accepted Mutation). These matrices help quantify the similarity between amino acids in an alignment.

Formula for Substitution Matrices Both BLOSUM and PAM matrices' elements are the log odds ratio of two amino acids i and j replacing each other in a homologous sequence compared to them replacing each other in two unrelated sequences. These matrices are calculated based on observed substitution frequencies, adjusted by background frequencies, and are scaled for practical application in sequence alignments. The formula for the scoring matrix elements can be described as:

$$M_{ij} = \left\lceil \lambda \log \left(\frac{p_{ij}}{q_i q_j} \right) \right\rceil$$

Where:

- p_{ij} is the observed probability of amino acid j being replaced by amino acid i in a given evolutionary distance.
- q_i and q_j are the background frequencies of amino acids i and j , respectively.
- λ , is an arbitrary factor, chosen to minimize rounding errors when converting to integers.

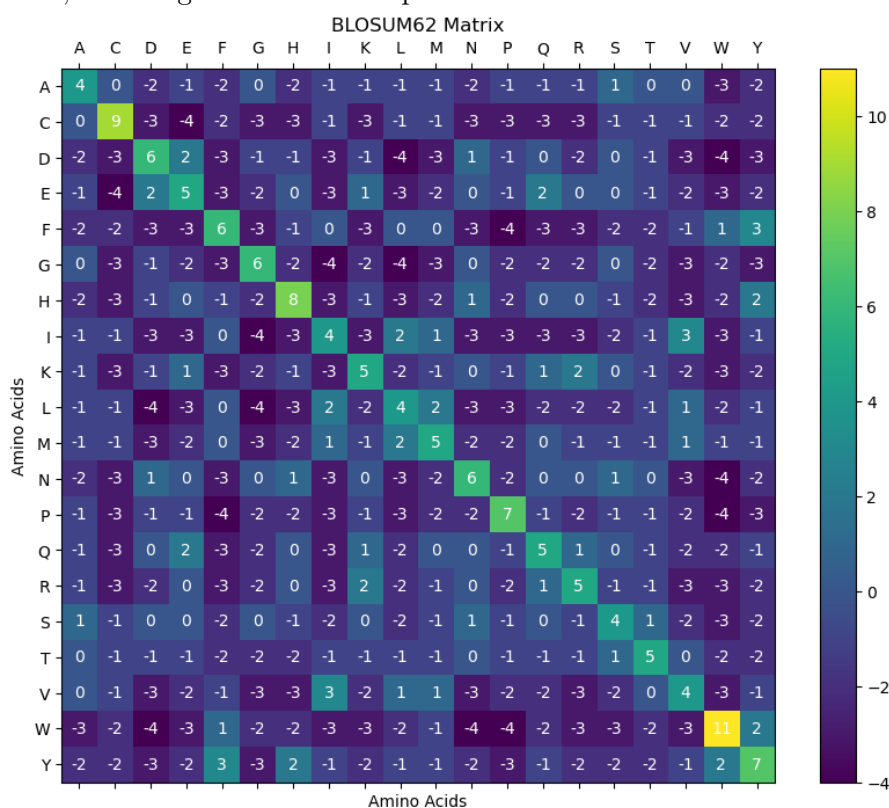
The role of scaling with λ : Rounding to Integers The transition from the theoretically derived logarithmic scores, which are real numbers, to practical integer scores used in alignment algorithms involves scaling by a factor of λ . The scaling factor λ is chosen such that the scaled scores remain proportional to the log-odds scores but are transformed into integers for practical use.

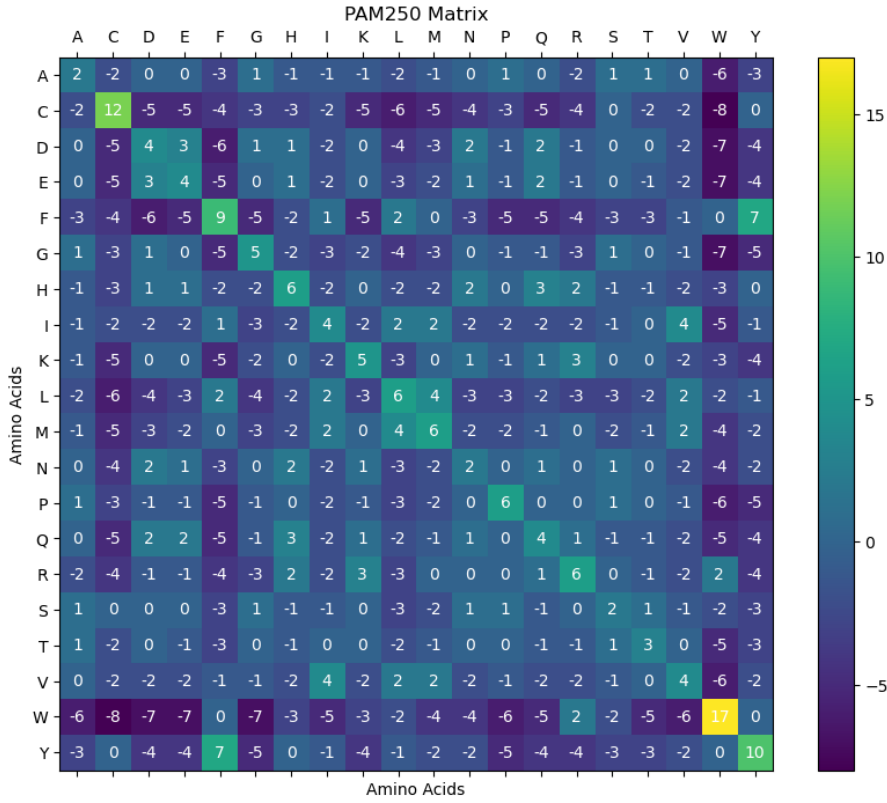
The use of integer values in scoring matrices like BLOSUM and PAM is indeed rooted in computational efficiency, particularly from the era when integer calculations were significantly faster than floating point calculations.

Understanding PAM Matrices PAM (Point Accepted Mutation) matrices measure the likelihood of one amino acid being replaced by another over evolutionary time Dayhoff et al. [1978]. A PAM1 matrix represents the probability of amino acid substitutions within 1% evolutionary divergence. Higher order PAM matrices (like PAM250) are derived by taking the matrix power of a basic PAM1 matrix to extrapolate to greater evolutionary distances.

Understanding BLOSUM Matrices BLOSUM (BLOCKS SUBstitution Matrix) matrices are based on observed substitutions in blocks of local alignments of protein sequences Henikoff and Henikoff [1992]. The number in a BLOSUM matrix refers to the percent-identity threshold used when *building* it: BLOSUM62, for instance, is constructed by clustering sequences that are at least 62% identical so that closely related sequences do not dominate the substitution counts. Lower-numbered matrices (e.g. BLOSUM45) are tuned for more divergent sequences, while higher-numbered ones (e.g. BLOSUM80) are tuned for more similar sequences.

Some example BLOSUM and PAM Matrices To illustrate the construction and use of these matrices, we will generate visual representations of BLOSUM62 and PAM250 using Python.





Estimating Probabilities q_i and p_{ij} from Alignments

Background Frequencies (q_i) The probability q_i represents the background frequency of amino acid i across a set of alignments or within a single alignment, depending on the context. It is estimated by counting the occurrence of each amino acid i in all alignments and then dividing by the total number of amino acid occurrences.

Estimation of $\hat{q}_i$: $\hat{q}_i = \frac{n_i}{N}$

where n_i is the number of times amino acid i appears in the alignments, and N is the total number of amino acid residues in all alignments.

Joint Probabilities (p_{ij}) The probability p_{ij} quantifies how often a particular substitution between amino acids i and j occurs. For symmetric matrices, the probabilities p_{ij} and p_{ji} are averaged since the direction of substitution does not impact the score.

Calculation of $\hat{p}_{ij}$: $\hat{p}_{ij} = \frac{f_{ij} + f_{ji}}{2T}$

where f_{ij} and f_{ji} are the frequencies of observing amino acids i and j substituted for one another in the alignments, and T is the total number of substitutions observed.

Estimation Process

1. **Collect Alignment Data:** Aggregate multiple protein sequence alignments that are representative of the evolutionary distances or functional similarities of interest.
2. **Count Occurrences and Substitutions:**
 - Count each amino acid's occurrence to estimate $\hat{q}_i$.
 - Count each substitution pair (both ij and ji) to determine f_{ij} and f_{ji} .
3. **Normalize and Average:**
 - Average the counts of f_{ij} and f_{ji} to reflect the symmetric nature of substitutions.

Example in Python Here's a Python snippet to calculate these probabilities from a mock alignment dataset. The example aims to describe the principle, and include a very limited number of alignments (1).

```
import numpy as np

# Input data as a list of tuples of strings (aligned sequences)
alignments = [
    ("IAMAPEPTIDE",
     "LAMVPDPWIDD")
]

# Flatten all characters into a single list for q_i calculation
all_characters = [char for seq in alignments for string in seq for char in string]

# Calculate q_i
unique, counts = np.unique(all_characters, return_counts=True)
q_i = dict(zip(unique, counts / counts.sum()))

# Calculate p_ij
pair_counts = {}
for seq1, seq2 in alignments:
    for i in range(min(len(seq1), len(seq2))): # Ensure index is within the shortest sequence
        pair = tuple(sorted([seq1[i], seq2[i]])) # Sort the pair to handle symmetry
        if pair in pair_counts:
            pair_counts[pair] += 1.0
        else:
            pair_counts[pair] = 1.0
# Normalize and ensure symmetry in the pair counts
for (a1, a2), count in list(pair_counts.items()):
    if a1 != a2:
        reversed_pair = (a2, a1)
        avg_count = pair_counts[(a1, a2)] / 2.0
        pair_counts[(a1, a2)] = avg_count
        pair_counts[reversed_pair] = avg_count

p_ij = {pair: count / sum(pair_counts.values()) for pair, count in pair_counts.items()}

print("Background Frequencies (q_i):\n" + "\n".join(f"{key}: {value:.3f}" for key, value in q_i.items()))
print()
print("Joint Probabilities (p_ij):\n" + "\n".join(f"{key}: {value:.3f}" for key, value in p_ij.items()))

Background Frequencies (q_i):
A: 0.136
D: 0.182
E: 0.091
I: 0.136
L: 0.045
M: 0.091
P: 0.182
T: 0.045
V: 0.045
W: 0.045
```

Joint Probabilities (p_{ij}):

(‘I’, ‘L’): 0.045
 (‘A’, ‘A’): 0.091
 (‘M’, ‘M’): 0.091
 (‘A’, ‘V’): 0.045
 (‘P’, ‘P’): 0.182
 (‘D’, ‘E’): 0.091
 (‘T’, ‘W’): 0.045
 (‘I’, ‘I’): 0.091
 (‘D’, ‘D’): 0.091
 (‘L’, ‘I’): 0.045
 (‘V’, ‘A’): 0.045
 (‘E’, ‘D’): 0.091
 (‘W’, ‘T’): 0.045

The Approximate Reasoning Behind Additive Log-Odds Scores Understanding the logic behind sums of log-odds scores for scoring sequence alignments, like the sums we get when scoring sequence alignments by BLOSUM or PAM scores, is quite straight forward. The primary goal is to assess the likelihood of sequences being homologous versus them occurring by random chance. This assessment is quantified using probabilities that compare alignments under the hypotheses of homology and non-homology.

To understand why additive log-odds scores, like the ones in BLOSUM and PAM matrices, are used for evaluating alignment quality, let’s delve into the logic step by step. The main goal is to assess how likely an alignment between two sequences is, under the hypothesis that they are homologous versus non-homologous.

We start the likelihood ratio of observing the alignment if the sequences are homologous to the probability of observing the same alignment if they are not homologous.

$$\frac{\text{Pr}(\text{Sequence alignment given the sequences are homologues})}{\text{Pr}(\text{Sequence alignment given the sequences are not homologues})}$$

If we assume that the alignment is made up of independent positions, we approximate this ratio as a product of the probabilities of each alignment position i :

$$\frac{\text{Pr}(\text{align pos } i | \text{homologues})}{\text{Pr}(\text{align pos } i | \text{not homologues})} \approx \prod_i \frac{\text{Pr}(a_i, b_i)}{\text{Pr}(a_i) \text{Pr}(b_i)}$$

Here, $\text{Pr}(a_i, b_i)$ is the probability of observing amino acids a_i and b_i paired together, while $\text{Pr}(a_i)$ and $\text{Pr}(b_i)$ are the independent probabilities of observing a_i and b_i respectively.

We can convert this product into a summation by observing the logarithm:

$$\log \left(\prod_i \frac{\text{Pr}(a_i, b_i)}{\text{Pr}(a_i) \text{Pr}(b_i)} \right) = \sum_i \log \left(\frac{\text{Pr}(a_i, b_i)}{\text{Pr}(a_i) \text{Pr}(b_i)} \right)$$

The log transformation simplifies the product into a sum, making it easier to compute and interpret. The resulting sum of logarithmic ratios:

$$\sum_i \log \left(\frac{\text{Pr}(a_i, b_i)}{\text{Pr}(a_i) \text{Pr}(b_i)} \right)$$

can be rewritten in terms of log-odds scores $d(a_i, b_i)$ for each position i . Apart from the scaling this also equates the log-odds ratios of scoring matrices, $M_{a_i b_i}$. So, the overall alignment score can be represented as:

$$C \sum_i M_{a_i b_i}$$

Rationale for Using Log-Odds Scores The use of log-odds scores is based on the following reasoning:

- **Quantitative Measure:** Log-odds scores provide a quantitative measure that reflects how much more likely an alignment is to occur if the sequences are homologous rather than just by random chance.
- **Additivity:** Since the log transformation converts products into sums, the scoring system becomes additive. This additivity is crucial because it allows the alignment score to be computed efficiently as the sum of independent position scores.

- **Biological Relevance:** The log-odds ratio considers not only the frequency of individual amino acids but also the frequency of their co-occurrence in homologous sequences versus their occurrence in non-homologous sequences. This method effectively captures evolutionary and functional relationships between proteins.

Conclusion By scoring alignments based on the sums of M_{a_i, b_i} , which are the log-odds scores for each position, bioinformatics tools can efficiently and effectively differentiate between sequences that are likely to be homologous and those that are not. This scoring mechanism is critical for the accurate detection and analysis of evolutionary relationships in sequence alignment tasks. This is the reason that we are summing up position-wise scores when using Needleman-Wunsch (instead of e.g. calculating their product).

References

Code: Aligning amino acid sequences

Protein sequences We can use the exact same alignment methods for protein sequences, as long as we define an appropriate scoring function. Here we will use a PAM250 matrix for the matches, and a gap penalty of 10.

```
PAM250 = {
'A': {'A': 2, 'C': -2, 'D': 0, 'E': 0, 'F': -3, 'G': 1, 'H': -1, 'I': -1, 'K': -1, 'L': -2,
'C': {'A': -2, 'C': 12, 'D': -5, 'E': -5, 'F': -4, 'G': -3, 'H': -3, 'I': -2, 'K': -5, 'L': -6,
'D': {'A': 0, 'C': -5, 'D': 4, 'E': 3, 'F': -6, 'G': 1, 'H': 1, 'I': -2, 'K': 0, 'L': -4,
'E': {'A': 0, 'C': -5, 'D': 3, 'E': 4, 'F': -5, 'G': 0, 'H': 1, 'I': -2, 'K': 0, 'L': -3,
'F': {'A': -3, 'C': -4, 'D': -6, 'E': -5, 'F': 9, 'G': -5, 'H': -2, 'I': 1, 'K': -5, 'L': 2,
'G': {'A': 1, 'C': -3, 'D': 1, 'E': 0, 'F': -5, 'G': 5, 'H': -2, 'I': -3, 'K': -2, 'L': -4,
'H': {'A': -1, 'C': -3, 'D': 1, 'E': 1, 'F': -2, 'G': -2, 'H': 6, 'I': -2, 'K': 0, 'L': -2,
'I': {'A': -1, 'C': -2, 'D': -2, 'E': -2, 'F': 1, 'G': -3, 'H': -2, 'I': 5, 'K': -2, 'L': 2,
'K': {'A': -1, 'C': -5, 'D': 0, 'E': 0, 'F': -5, 'G': -2, 'H': 0, 'I': -2, 'K': 5, 'L': -3,
'L': {'A': -2, 'C': -6, 'D': -4, 'E': -3, 'F': 2, 'G': -4, 'H': -2, 'I': 2, 'K': -3, 'L': 6,
'M': {'A': -1, 'C': -5, 'D': -3, 'E': -2, 'F': 0, 'G': -3, 'H': -2, 'I': 2, 'K': 0, 'L': 4,
'N': {'A': 0, 'C': -4, 'D': 2, 'E': 1, 'F': -3, 'G': 0, 'H': 2, 'I': -2, 'K': 1, 'L': -3,
'P': {'A': 1, 'C': -3, 'D': -1, 'E': -1, 'F': -5, 'G': 0, 'H': 0, 'I': -2, 'K': -1, 'L': -3,
'Q': {'A': 0, 'C': -5, 'D': 2, 'E': 2, 'F': -5, 'G': -1, 'H': 3, 'I': -2, 'K': 1, 'L': -2,
'R': {'A': -2, 'C': -4, 'D': -1, 'E': -1, 'F': -4, 'G': -3, 'H': 2, 'I': -2, 'K': 3, 'L': -3,
'S': {'A': 1, 'C': 0, 'D': 0, 'E': 0, 'F': -3, 'G': 1, 'H': -1, 'I': -1, 'K': 0, 'L': -3,
'T': {'A': 1, 'C': -2, 'D': 0, 'E': 0, 'F': -3, 'G': 0, 'H': -1, 'I': 0, 'K': 0, 'L': -2,
'V': {'A': 0, 'C': -2, 'D': -2, 'E': -2, 'F': -1, 'G': -1, 'H': -2, 'I': 4, 'K': -2, 'L': 2,
'W': {'A': -6, 'C': -8, 'D': -7, 'E': -7, 'F': 0, 'G': -7, 'H': -3, 'I': -5, 'K': -3, 'L': -2,
'Y': {'A': -3, 'C': 0, 'D': -4, 'E': -4, 'F': 7, 'G': -5, 'H': 0, 'I': -1, 'K': -4, 'L': -1

def gap_penalty():
    return -10.0

def match_score(letterA,letterB):
    if letterA == '-' or letterB == '-':
        return gap_penalty()
    else:
        return PAM250[letterA][letterB]

seqA,seqB = global_align("IAMAPEPTIDE","IAMPEPPED",True)
print_alignment(seqA,seqB)

-   I   A   M   P   E   P   P   E   D
-   0.0 -10.0 -20.0 -30.0 -40.0 -50.0 -60.0 -70.0 -80.0 -90.0
I -10.0  5.0 -5.0 -15.0 -25.0 -35.0 -45.0 -55.0 -65.0 -75.0
A -20.0 -5.0  7.0 -3.0 -13.0 -23.0 -33.0 -43.0 -53.0 -63.0
M -30.0 -15.0 -3.0 13.0  3.0 -7.0 -17.0 -27.0 -37.0 -47.0
A -40.0 -25.0 -13.0  3.0 14.0  4.0 -6.0 -16.0 -26.0 -36.0
P -50.0 -35.0 -23.0 -7.0  9.0 13.0 10.0  0.0 -10.0 -20.0
E -60.0 -45.0 -33.0 -17.0 -1.0 13.0 12.0  9.0  4.0 -6.0
P -70.0 -55.0 -43.0 -27.0 -11.0  3.0 19.0 18.0  8.0  3.0
T -80.0 -65.0 -53.0 -37.0 -21.0 -7.0  9.0 19.0 18.0  8.0
I -90.0 -75.0 -63.0 -47.0 -31.0 -17.0 -1.0  9.0 17.0 16.0
D -100.0 -85.0 -73.0 -57.0 -41.0 -27.0 -11.0 -1.0 12.0 21.0
E -110.0 -95.0 -83.0 -67.0 -51.0 -37.0 -21.0 -11.0  3.0 15.0

Best score: 15.0
```

IAMAPEPTIDE
IAM -PEPP -ED

```
seqA,seqB = global_align("MYPERFECTDAY","THERESAMAY",False)
print_alignment(seqA,seqB)
```

Best score: 3.0
MYPERFECTDAY
T -HER -ESAMAY

```
seqA,seqB = local_align("MYPERFECTDAY","THERESAMAY",False)
print_alignment(seqA,seqB)
```

Best score: 14.0
PERFECTDAY
HER -ESAMAY

0.1.4 Multiple Sequence Alignments

Multiple Sequence Alignments

Introduction While pairwise alignments enable comparisons between two sequences, many biological analyses require a more comprehensive approach: the alignment of multiple sequences simultaneously. Multiple sequence alignments (MSAs) offer a holistic view of sequence similarities and differences, illuminating functional and evolutionary relationships across diverse genomes and proteomes.

Exponential Time Complexity In principle, one can use dynamic programming to form MSAs. This can be achieved by extending the ideas used for pairwise alignments, expanding the dynamic programming matrix into a tensor with as many dimensions as there are sequences. However, a major drawback is the exponential time complexity. Aligning T sequences, each of length N , results in a time complexity of $O(N^T)$, making it impractical for datasets with more than a few sequences.

Iterative Approaches as a Practical Alternative Given the inefficiency of dynamic programming for MSAs, bioinformaticians have turned to heuristics that approximate the behavior of full dynamic programming. In particular, iterative approaches such as progressive alignment simplify the alignment process by breaking it down into manageable steps. Progressive alignment, for instance, first evaluates the pairwise similarities between sequences by computing their pairwise alignments, then iteratively aligns sequences according to which are most similar. This reduces the computational burden and allows for the processing of large datasets in a reasonable time frame.

Progressive Alignments While dynamic programming offers an optimal solution for pairwise alignments, its inefficiency for MSAs necessitates a balance between accuracy and computational efficiency. Iterative and heuristic methods like progressive alignment and consistency-based algorithms strike this balance, providing near-optimal solutions in significantly less time.

A key method for constructing MSAs is progressive alignment, a stepwise approach that builds upon pairwise alignments to create a broader, more nuanced comparison. The progressive alignment method begins by generating a similarity matrix (or distance matrix if we instead quantify how different the sequences are), which quantifies the pairwise similarity between all sequences in the dataset.

Progressive alignment iteratively aligns sequences in order of their proximity on the tree. This process begins with the closest pair of sequences, which are aligned directly. Subsequent sequences are added progressively, with each new sequence being aligned to the growing consensus. This stepwise process creates an alignment that reflects both sequence similarities and evolutionary relationships.

The progressive alignment method offers several advantages. It efficiently handles large datasets, making it well-suited for genomics and proteomics research. By utilizing an evolutionary framework, it provides deeper insights into the conserved and divergent features of sequences, aiding in the identification of functionally important regions and evolutionary patterns.

In this chapter, we delve into the mechanics of progressive alignment, exploring its implementation and applications in bioinformatics. By understanding this approach, we gain valuable insights into the biological relationships and evolutionary trajectories that shape our world.

Progressive Alignment Steps The process builds alignments in a stepwise manner, starting with the most similar sequences and gradually incorporating others. Here's a detailed look at how to create a progressive alignment. Let's assume you start with T sequences, $C = \{A_1, \dots, A_T\}$.

1. **Compute Pairwise Alignments:** Calculate all pairwise alignments between each pair of sequences in C using a suitable algorithm such as Needleman-Wunsch or Smith-Waterman. This provides a matrix of scores or distances that indicate how similar or different each pair of sequences is.
2. **Use the pairwise distances to identify the two sequences in C , A_i and A_j , that are closest** (those with the highest similarity score or lowest distance), and form an alignment of those two sequences. Remove A_i and A_j from C . We call the alignment of those two sequences B .

3. Add B to C . Set the score or distance from B to each of the other sequences in C to the average of their scores or distances from A_i and A_j .
4. If there is more than one sequence left in C , repeat from step 2.

Progressive Alignment Example

Applications of Progressive Alignments Modern bioinformatics tools, such as MUSCLE Edgar [2004], MAFFT Katoh et al. [2002], T-Coffee Notredame et al. [2000], and Kalign Lassmann and Sonnhammer [2005], incorporate iterative refinement as a core feature, allowing researchers to achieve high-quality alignments with minimal manual intervention. These tools use sophisticated algorithms to iteratively refine alignments, often integrating additional information such as structural data or evolutionary constraints. This integration enhances the overall robustness of the alignment process, making these tools highly effective for a wide range of applications in genomics, proteomics, and evolutionary biology.

Iterative Refinement of MSAs While progressive alignment provides a structured approach to generating multiple sequence alignments (MSAs), it is often just the starting point for more refined alignments. The initial alignment produced by progressive methods can be improved through iterative refinement, a crucial step for enhancing alignment accuracy and resolving inconsistencies that may arise during the initial process.

The Need for Iterative Refinement Progressive alignment can introduce biases, particularly when the initial pairwise alignments are not optimal or when sequences with varying evolutionary distances are involved. These biases can propagate through the alignment process, leading to suboptimal alignments, especially in regions where the evolutionary signals are weak or ambiguous. Iterative refinement addresses these issues by revisiting and adjusting the alignment, ensuring that it more accurately reflects the true evolutionary relationships and functional similarities among the sequences.

Iterative Refinement Techniques There are several approaches to iterative refinement, each with its own strengths and applications:

- **Realignment of Subsets:** One common method involves selecting subsets of sequences or regions of the alignment and realigning them independently. This approach is particularly useful for correcting local misalignments that occur due to the progressive alignment process. By focusing on smaller sections of the alignment, the algorithm can correct errors without the computational burden of realigning the entire sequence set.
- **Profile-based Realignment:** In this technique, the MSA is treated as a series of profiles, where each profile represents a group of already aligned sequences. The iterative refinement process involves realigning these profiles against each other or introducing new sequences into the existing profiles. This method is effective in maintaining the overall structure of the alignment while correcting specific errors.
- **Consistency-based Refinement:** This approach leverages consistency information, ensuring that the aligned pairs in the final MSA are consistent with the pairwise alignments. By re-evaluating the alignment through the lens of consistency, this method can reduce the occurrence of alignment errors that arise from conflicts between pairwise comparisons and the global alignment.
- **Iterative Re-scoring and Re-alignment:** Another approach involves re-scoring the alignment based on a specific objective function (such as a scoring matrix or evolutionary model) and then realigning the sequences to maximize this score. This iterative process continues until the alignment stabilizes and no further improvements can be made. This method ensures that the alignment converges to a local optimum that best fits the given scoring criteria.

Applications and Advantages of Iterative Refinement Iterative refinement is widely used in bioinformatics tools such as MUSCLE Edgar [2004] and MAFFT Katoh et al. [2002], which implement various refinement strategies to enhance MSA quality. The iterative process ensures that the final alignment is more accurate and reliable, making it particularly valuable for downstream analyses, such as phylogenetic tree construction, structural modeling, and functional annotation.

Moreover, iterative refinement is adaptable, allowing bioinformaticians to tailor the process based on the specific characteristics of the sequences being aligned. This flexibility makes it an indispensable tool in the alignment of sequences with complex evolutionary histories, such as those involving duplications, deletions, or recombination events.

References

Code for Progressive multiple sequence alignments

Here we will implement one type of multiple sequence alignment, the Progressive multiple sequence alignment.

We build our algorithm on top of the previous code for pairwise alignment, the Needleman-Wunsch algorithm. We hence begin with copying the code from previous example. However, we make edits so that we are pairwise aligning lists of sequences, instead of single sequences against single sequences.

First we need to modify our score function so that it can take multiple sequences in account. Here we elect to score the sequences according to their average PAM250 score. To do so, we generalize the concept of a “sequences” to be a list of one or more strings, representing either a single sequence (when there is one element in the list) or alignments of sequences (when there are multiple sequences present).

```
gap_penalty = -1.0
```

```
def match_score_pam(sequencesA,posA, sequencesB,posB):
    score, n = 0.,0
    for seqA in sequencesA:
        for seqB in sequencesB:
            score += PAM250[seqA[posA]][seqB[posB]]
            n += 1
    return score/n
```

```
PAM250 = {
```

```
'A': {'A': 2, 'C': -2, 'D': 0, 'E': 0, 'F': -3, 'G': 1, 'H': -1, 'I': -1, 'K': -1, 'L': -2,
'C': {'A': -2, 'C': 12, 'D': -5, 'E': -5, 'F': -4, 'G': -3, 'H': -3, 'I': -2, 'K': -5, 'L': -6,
'D': {'A': 0, 'C': -5, 'D': 4, 'E': 3, 'F': -6, 'G': 1, 'H': 1, 'I': -2, 'K': 0, 'L': -4,
'E': {'A': 0, 'C': -5, 'D': 3, 'E': 4, 'F': -5, 'G': 0, 'H': 1, 'I': -2, 'K': 0, 'L': -3,
'F': {'A': -3, 'C': -4, 'D': -6, 'E': -5, 'F': 9, 'G': -5, 'H': -2, 'I': 1, 'K': -5, 'L': 2,
'G': {'A': 1, 'C': -3, 'D': 1, 'E': 0, 'F': -5, 'G': 5, 'H': -2, 'I': -3, 'K': -2, 'L': -4,
'H': {'A': -1, 'C': -3, 'D': 1, 'E': 1, 'F': -2, 'G': -2, 'H': 6, 'I': -2, 'K': 0, 'L': -2,
'I': {'A': -1, 'C': -2, 'D': -2, 'E': -2, 'F': 1, 'G': -3, 'H': -2, 'I': 5, 'K': -2, 'L': 2,
'K': {'A': -1, 'C': -5, 'D': 0, 'E': 0, 'F': -5, 'G': -2, 'H': 0, 'I': -2, 'K': 5, 'L': -3,
'L': {'A': -2, 'C': -6, 'D': -4, 'E': -3, 'F': 2, 'G': -4, 'H': -2, 'I': 2, 'K': -3, 'L': 6,
'M': {'A': -1, 'C': -5, 'D': -3, 'E': -2, 'F': 0, 'G': -3, 'H': -2, 'I': 2, 'K': 0, 'L': 4,
'N': {'A': 0, 'C': -4, 'D': 2, 'E': 1, 'F': -3, 'G': 0, 'H': 2, 'I': -2, 'K': 1, 'L': -3,
'P': {'A': 1, 'C': -3, 'D': -1, 'E': -1, 'F': -5, 'G': 0, 'H': 0, 'I': -2, 'K': -1, 'L': -3,
'Q': {'A': 0, 'C': -5, 'D': 2, 'E': 2, 'F': -5, 'G': -1, 'H': 3, 'I': -2, 'K': 1, 'L': -2,
'R': {'A': -2, 'C': -4, 'D': -1, 'E': -1, 'F': -4, 'G': -3, 'H': 2, 'I': -2, 'K': 3, 'L': -3,
'S': {'A': 1, 'C': 0, 'D': 0, 'E': 0, 'F': -3, 'G': 1, 'H': -1, 'I': -1, 'K': 0, 'L': -3,
'T': {'A': 1, 'C': -2, 'D': 0, 'E': 0, 'F': -3, 'G': 0, 'H': -1, 'I': 0, 'K': 0, 'L': -2,
'V': {'A': 0, 'C': -2, 'D': -2, 'E': -2, 'F': -1, 'G': -1, 'H': -2, 'I': 4, 'K': -2, 'L': 2,
'W': {'A': -6, 'C': -8, 'D': -7, 'E': -7, 'F': 0, 'G': -7, 'H': -3, 'I': -5, 'K': -3, 'L': -2,
'Y': {'A': -3, 'C': 0, 'D': -4, 'E': -4, 'F': 7, 'G': -5, 'H': 0, 'I': -1, 'K': -4, 'L': -1,
'-': {'A': 0, 'C': 0, 'D': 0, 'E': 0, 'F': 0, 'G': 0, 'H': 0, 'I': 0, 'K': 0, 'L': 0, 'M':
```

Similarly, we modify our code for tracing the dynamic programming matrix, so that it can handle multiple sequences.

```
def format_msa(seqA,seqB,trace):
    outA,outB = [" " for s in seqA],[" " for s in seqB]
    i,j = len(seqA[0]),len(seqB[0])
    while i>0 or j>0:
        di,dj = trace[i,j]
        i += int(di)
```

```

    j += int(dj)
    if di == 0:
        for ix in range(len(seqA)):
            outA[ix] = " -" + outA[ix]
    else:
        for ix in range(len(seqA)):
            outA[ix] = seqA[ix][i] + outA[ix]
    if dj == 0:
        for ix in range(len(seqB)):
            outB[ix] = " -" + outB[ix]
    else:
        for ix in range(len(seqB)):
            outB[ix] = seqB[ix][j] + outB[ix]
    return outA,outB

```

Next, we copy the code for the pairwise alignment from the previous exercise

```

import argparse
import numpy as np

def align(seqA, seqB, print_dynamic_matrix = False, score = match_score_pam):
    # Initiating variables
    m, n = len(seqA[0])+1, len(seqB[0])+1
    S = np.zeros((m,n))
    trace = np.zeros((m,n,2))
    # Set up dynamic programming matrices
    S[0,0] = 0.
    trace[0,0,:] = (0.,0.)
    for i in range(1,m):
        S[i,0] = gap_penalty * i
        trace[i,0,:] = (-1.,0.)
    for j in range(1,n):
        S[0,j] = gap_penalty * j
        trace[0,j,:] = (0., -1.)
    # Set up dynamic programming matrices
    for i in range(1,m):
        for j in range(1,n):
            match = S[i -1][j -1] + score(seqA,i -1,seqB,j -1)
            delete = S[i -1][j] + gap_penalty
            insert = S[i][j -1] + gap_penalty
            S[i,j] = max(match, delete, insert)
            if match >= max(delete,insert):
                trace[i,j,:] = (-1, -1.)
            elif delete >= insert:
                trace[i,j,:] = (-1,0)
            else:
                trace[i,j,:] = (0, -1)
    if print_dynamic_matrix:
        print_dynamic(seqA,seqB,S)
    return S[m -1,n -1],format_msa(seqA,seqB,trace)

```

So far we just modified our pairwise alignment code. Lets turn to the code building our progressive alignment.

```

def align_all(seqs, debug = False):
    sequences = [[s] for s in seqs]
    while len(sequences)>1:
        k = len(sequences)
        scores = np.zeros((k -1,k))
        # Align all sequences against each other
        for a in range(k -1):
            for b in range(a+1,k):
                score,alignment = align(sequences[a],sequences[b])
                scores[a,b -1] = score
        # Find the pair of sequences with highest alignment score
        i,j = np.unravel_index(scores.argmax(),scores.shape)
        j += 1
        # Realign the sequences (we only stored the scores, not the alignment)
        score,(a1,a2) = align(sequences[i],sequences[j])
        # Replace sequence i with the pairwise alignment of i and j
        sequences[i] = a1 + a2
        del sequences[j]
        if debug:
            print("The algorithm aligned")
            for s in a1:
                print (s)
            print("to")
            for s in a2:
                print(s)
            print()
    return sequences[0]

```

Lets try this code for a couple of peptides.

```

seqs = align_all(["IAMAPEPTIDE","IAMPEPTIDE","IAMPEPPED","IAMAPEPPERD"],True)
for s in seqs:
    print (s)

```

```

The algorithm aligned
IAMAPEPTIDE
to
IAM -PEPTIDE

```

```

The algorithm aligned
IAM -PEPPE -D
to
IAMAPEPPERD

```

```

The algorithm aligned
IAMAPEPTID -E
IAM -PEPTID -E
to
IAM -PEPP -E -D
IAMAPEPP -ERD

```

```

IAMAPEPTID -E
IAM -PEPTID -E
IAM -PEPP -E -D
IAMAPEPP -ERD

```

Lets try the strategy for something slightly more complex. Here is an alignment of Insulin precursors from Human, Macaque, Dog and Mouse.

```
seqs = align_all(["MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDLQVGGVELGSSQEPGAG
                  "MALWMRLLPLLALLALWGPDPAPAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDPQVGGVELGSSQEPGAG
                  "MALWMRLLPLLALLALWAPAPTRAFVNQHLCGSHLVEALYLVCGERGFFYTPKARREVEDLQVRDVELAGAPGEG
                  "MALLVHFLPLLALLALWEPKPTQAFVKQHLCPHLVEALYLVCGERGFFYTPKSRREVEDPQVEQLGSSQEPGDL

for s in seqs:
    print(s)
```

The algorithm aligned

```
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAED -LQVGGVELGSSQEPGAGSLQPLALEGSLQKRGIV
to
MALWMRLLPLLALLALWGPDPAPAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDP -QVGGVELGSSQEPGAGSLQPLALEGSLQKRGIV
```

The algorithm aligned

```
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAED -LQV -GQVELGSSQEPGAGSLQPLALEGSLQKRG
MALWMRLLPLLALLALWGPDPAPAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDP -QV -GQVELGSSQEPGAGSLQPLALEGSLQKRG
to
MALWMRLLPLLALLALWAPAPTRAFVNQHLCGSHLVEALYLVCGERGFFYTPKARREVED -LQVR -DVELAGAPGEGGLQPLALEGALQKRG
```

The algorithm aligned

```
MALWM -RLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAED -LQV -GQVELGSSQEPGAGSLQPLALE -GSL
MALWM -RLLPLLALLALWGPDPAPAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDP -QV -GQVELGSSQEPGAGSLQPLALE -GSL
MALWM -RLLPLLALLALWAPAPTRAFVNQHLCGSHLVEALYLVCGERGFFYTPKARREVED -LQVR -DVELAGAPGEGGLQPLALE -GAL
to
MAL -LVHFLPLLALLALWEPKPTQAFVKQHLCPHLVEALYLVCGERGFFYTPKSRREVEDP -QV -EQLELGSSQEP -D -LQTLALEVAR
```

```
MALWM -RLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAED -LQV -GQVELGSSQEPGAGSLQPLALE -GSL
MALWM -RLLPLLALLALWGPDPAPAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAEDP -QV -GQVELGSSQEPGAGSLQPLALE -GSL
MALWM -RLLPLLALLALWAPAPTRAFVNQHLCGSHLVEALYLVCGERGFFYTPKARREVED -LQVR -DVELAGAPGEGGLQPLALE -GAL
MAL -LVHFLPLLALLALWEPKPTQAFVKQHLCPHLVEALYLVCGERGFFYTPKSRREVEDP -QV -EQLELGSSQEP -D -LQTLALEVAR
```

Sequence Logos

When studying multiple sequence alignments (MSAs), we often want to summarize patterns across many sequences rather than display each one individually. **Sequence logos** offer a compact visual representation: each position in the alignment is shown as a stack of letters, where the total height of the stack reflects how conserved that position is, and the height of each individual letter reflects how often that character appears.

Sequence logos were introduced by Schneider and Stephens [1990] and have since become a standard tool in bioinformatics for visualizing regulatory motifs, binding sites, and any other position-specific pattern in biological sequences.

Information Content The height of each column in a sequence logo is a quantitative measure called **information content**, derived from information theory.

Shannon Entropy For position i in the alignment, the **Shannon entropy** measures the uncertainty in the character distribution:

$$H_i = - \sum_a f_{i,a} \log_2 f_{i,a} \quad (6)$$

where $f_{i,a}$ is the observed frequency of character a at position i . Entropy is zero when every sequence has the same character (complete conservation) and is maximal when all characters appear with equal frequency (maximum uncertainty).

Information Content in Bits The **information content** at position i is the reduction in uncertainty relative to the maximum possible entropy:

$$I_i = \log_2(N) - H_i \quad (7)$$

where N is the alphabet size. This gives the following upper limits:

Alphabet	Size N	Maximum $I_{\max}$
DNA / RNA	4	$\log_2(4) = 2.00$ bits
Amino acids	20	$\log_2(20) \approx 4.32$ bits

A fully conserved position contributes 2 bits (DNA) or ≈ 4.32 bits (protein); a completely random position contributes 0 bits. Within each column, the height of an individual letter for character a is $I_i \times f_{i,a}$, so more frequent characters occupy a larger fraction of the column.

The higher maximum for amino acid logos reflects the larger alphabet: a conserved protein position carries more than twice the information of a conserved nucleotide position.

Examples We use the `logomaker` package Tareen and Kinney [2020] to generate sequence logos in Python.

```
import logomaker
import matplotlib.pyplot as plt
```

The Kozak Consensus Sequence The **Kozak sequence** is a short conserved motif surrounding the AUG start codon in eukaryotic mRNAs, recognised by the ribosome during translation initiation. Described by Marilyn Kozak [1987], the consensus is often written (GCC)**R**CCAUGG, where R is a purine (A or G).

Two positions have the strongest effect on translation efficiency:

- **Position -3** (three nucleotides before the A of AUG): a purine is strongly preferred, present in over 80 % of vertebrate mRNAs.

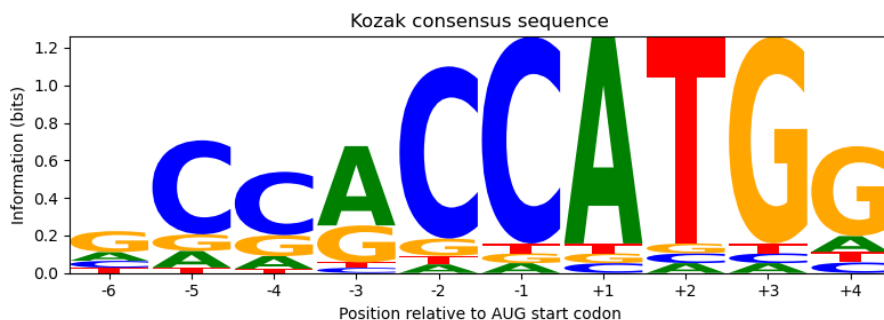
- **Position +4** (the nucleotide immediately after the AUG codon): G is preferred in about 74 % of vertebrate transcripts.

The following 20 sequences show representative Kozak contexts from vertebrate mRNAs, spanning six nucleotides upstream to four nucleotides downstream of the AUG.

```
kozak_seqs = [
    "GCCACCATGG", # canonical Kozak (A at -3, G at +4)
    "GCCGCCATGG", # G at -3
    "ACCACCATGG",
    "CCCACCATGG",
    "TCCACCATGG",
    "GCAACCATGG",
    "GCGACCATGG",
    "GGCGCCATGG", # G at -3
    "GCGGCCATGG", # G at -3
    "AAGACCATGG",
    "CGCGCCATGG", # G at -3
    "AACACCATGG",
    "CCGACCATGG",
    "ACCGCCATGG", # G at -3
    "GCCACCATGA", # A at +4 (weaker context)
    "GCCACCATGC", # C at +4 (weaker context)
    "GCCACCATGT", # T at +4 (weaker context)
    "GCAACCATGA",
    "TCCGCCATGG", # G at -3
    "GCCAGCATGG",
]
```

```
kozak_matrix = logomaker.alignment_to_matrix(kozak_seqs, to_type='information')
```

```
fig, ax = plt.subplots(figsize=(8, 3))
logomaker.Logo(kozak_matrix, ax=ax, color_scheme='classic')
ax.set_xticks(range(10))
ax.set_xticklabels([' -6', ' -5', ' -4', ' -3', ' -2', ' -1', ' +1', ' +2', ' +3', ' +4'])
ax.set_xlabel('Position relative to AUG start codon')
ax.set_ylabel('Information (bits)')
ax.set_title('Kozak consensus sequence')
plt.tight_layout()
plt.show()
```



The logo clearly shows the nearly invariant **AUG** at positions +1/+2/+3, the strong **purine** preference at -3, the **G** preference at +4, and relatively little conservation at the remaining positions — exactly what a biotech researcher considers when designing an expression construct.

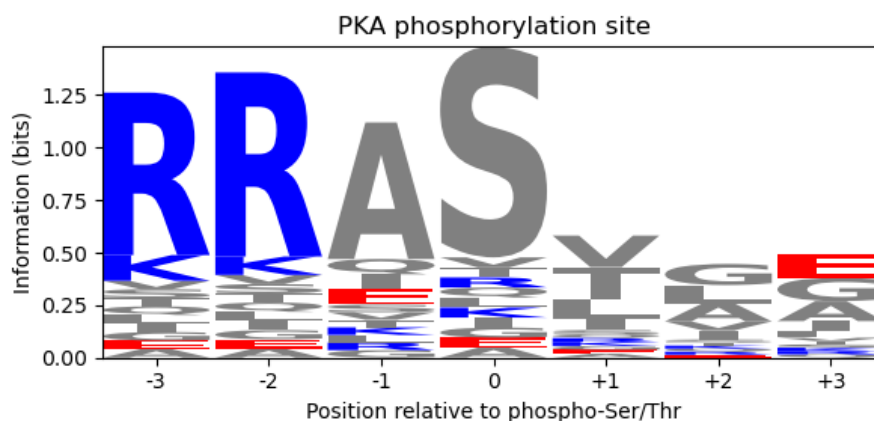
PKA Phosphorylation Site Protein kinase A (PKA) phosphorylates serine or threonine residues in target proteins. The kinase recognises a characteristic sequence motif: two basic residues (almost always Arg, occasionally Lys) appear two and three positions upstream of the phospho-Ser/Thr, giving the consensus **R-R-x-S/T** Songyang et al. [1994].

The following sequences show 7 residues centred on the phosphorylatable residue at position 0, drawn from experimentally validated PKA substrates.

```
pka_seqs = [
    "RRASLGA", # Kemptide -like
    "RRASVAI",
    "RRASTLG", # Thr at P0
    "RRESVAE", # Glu at P -1
    "RRASLGE",
    "KRASLGA", # Lys at P -3
    "RRQSTVG", # Gln at P -1, Thr at P0
    "RRASTIA", # Thr at P0
    "RRASVIG",
    "RRISVAI", # Ile at P -1
    "KRASTLG", # Lys at P -3, Thr at P0
    "RRASILA",
    "RRASTVE", # Thr at P0
    "RRASVAG",
    "RRASLGE",
    "RRASTLA", # Thr at P0
    "RRASIGE",
    "RRASVGE",
    "RKASTLG", # Lys at P -2, Thr at P0
    "RRASVAL",
]
```

```
pka_matrix = logomaker.alignment_to_matrix(pka_seqs, to_type='information')
```

```
fig, ax = plt.subplots(figsize=(6, 3))
logomaker.Logo(pka_matrix, ax=ax, color_scheme='charge')
ax.set_xticks(range(7))
ax.set_xticklabels([' -3', ' -2', ' -1', ' 0', ' +1', ' +2', ' +3'])
ax.set_xlabel('Position relative to phospho -Ser/Thr')
ax.set_ylabel('Information (bits)')
ax.set_title('PKA phosphorylation site')
plt.tight_layout()
plt.show()
```



The logo highlights the strongly conserved **Arg–Arg** doublet at positions -3 and -2 (tall blue letters — positive charge), the phospho-**S/T** at position 0 where both options are visible, and the largely variable positions elsewhere.

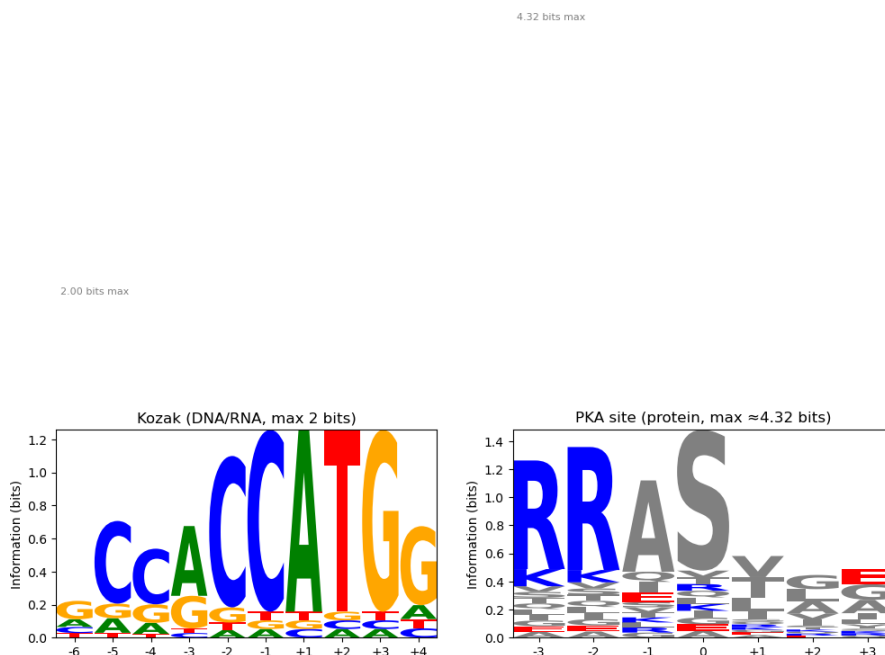
```
fig, axes = plt.subplots(1, 2, figsize=(12, 3))

# Kozak logo with DNA maximum marked
logomaker.Logo(kozak_matrix, ax=axes[0], color_scheme='classic')
axes[0].axhline(2.0, color='grey', linestyle='--', linewidth=0.8)
axes[0].text(0.01, 2.08, '2.00 bits max', transform=axes[0].get_yaxis_transform(),
            fontsize=8, color='grey')
axes[0].set_xticks(range(10))
axes[0].set_xticklabels(['-6', '-5', '-4', '-3', '-2', '-1', '+1', '+2', '+3', '+4'])
axes[0].set_ylabel('Information (bits)')
axes[0].set_title('Kozak (DNA/RNA, max 2 bits)')

# PKA logo with protein maximum marked
logomaker.Logo(pka_matrix, ax=axes[1], color_scheme='charge')
axes[1].axhline(4.32, color='grey', linestyle='--', linewidth=0.8)
axes[1].text(0.01, 4.40, '4.32 bits max', transform=axes[1].get_yaxis_transform(),
            fontsize=8, color='grey')
axes[1].set_xticks(range(7))
axes[1].set_xticklabels(['-3', '-2', '-1', '0', '+1', '+2', '+3'])
axes[1].set_ylabel('Information (bits)')
axes[1].set_title('PKA site (protein, max \approx 4.32 bits)')

plt.tight_layout()
plt.show()

/tmp/ipykernel_2106903/39490927.py:23: UserWarning: Tight layout not applied. The bottom and t
plt.tight_layout()
```



Comparing the two logos, note how the conserved R–R in the PKA site approaches its alphabet maximum (≈ 4.32 bits) while the most conserved nucleotide positions in the Kozak sequence are bounded by 2 bits. The dashed lines make these theoretical ceilings explicit.

Profile Hidden Markov Models

Overview Profile Hidden Markov Models (HMMs) are statistical models designed to capture the variability and patterns found in multiple sequence alignments (MSAs). They offer a robust way to model conserved regions, sequence motifs, and evolutionary variations, making them an essential tool in bioinformatics for sequence analysis and comparison.

Profile HMMs as representations of MSAs A multiple sequence alignment of a homologous family of protein domains reveals patterns of site-specific evolutionary conservation:

- **Conserved Residues:** Certain positions show high conservation, reflecting key functional or structural roles.
- **Substitution Patterns:** Some positions tolerate substitutions that conserve physiochemical properties like hydrophobicity, charge, or size.
- **Variable Positions:** Some positions exhibit near-neutral variability.
- **Insertions and Deletions:** Insertions and deletions are tolerated at some positions more than others.

A profile HMM is a position-specific scoring model that describes which symbols are likely to be observed and how frequently insertions or deletions occur at each position (column) of a multiple sequence alignment.

- **Probabilistic Basis:** HMMs have a formal probabilistic basis, allowing the use of probability theory to set and interpret the large number of free parameters in a profile, including position-specific gap and insertion parameters.
- **Automatable Methods:** These methods are mathematically consistent and automatable, enabling the creation of libraries of hundreds of profile HMMs for large-scale genome analysis.

Applications Profile HMMs offer several key applications in bioinformatics:

1. **Sequence Alignment:** By comparing a new sequence to a profile HMM, we can identify which regions of the sequence correspond to conserved regions in the MSA, helping to align the new sequence with known homologs.
2. **Domain Detection:** Profile HMMs can identify and characterize functional domains in proteins by matching sequence segments to domain profiles.
3. **Sequence Classification:** By comparing a sequence to various profile HMMs, we can classify it into families or groups, aiding in the identification of evolutionary relationships. One notable database of protein domain models is Pfam. Pfam and the HMMER software suite have been developed in parallel, providing comprehensive resources for protein families based on seed alignments.

Structure of a Profile HMM A Profile HMM is structured to mirror the columns of an MSA, with three main states for each column:

1. **Match States (M):** Represent conserved positions in the alignment, corresponding to columns with similar residues or nucleotides across multiple sequences. Match states store probabilities for each possible amino acid or nucleotide at that position, reflecting the distribution observed in the MSA.
2. **Insertion States (I):** Account for potential insertions between columns, capturing variability by allowing additional residues or nucleotides to be added. These states store probabilities for each possible amino acid or nucleotide to be inserted.
3. **Deletion States (D):** Represent positions where sequences may lack a corresponding residue or nucleotide, resulting in a gap in the alignment. Deletion states enable the model to handle varying lengths in sequences.

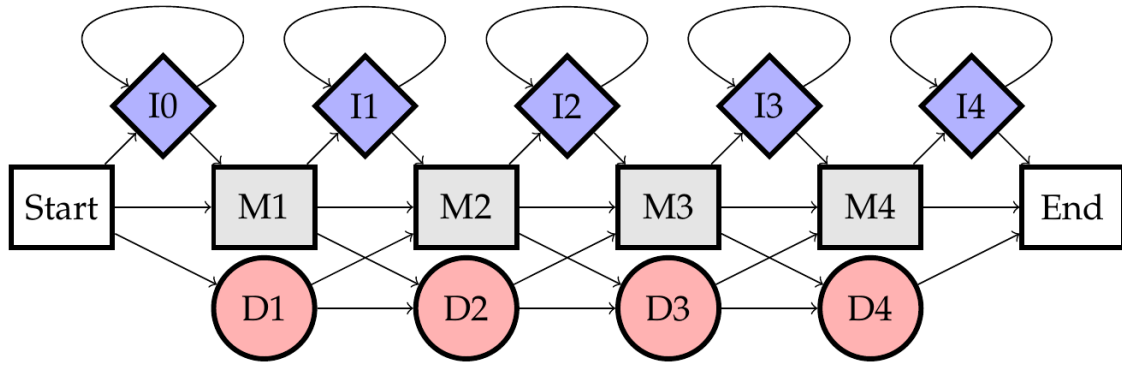


Figure 9: A state diagram of a profile HMM with 4 match states.

Emission Probabilities Emission probabilities in a Profile HMM quantify the likelihood of observing a specific amino acid or nucleotide at each position in an alignment. These probabilities are stored within match, insertion states (delete states are silent and don't emit sequence) and are derived from the frequencies of residues or nucleotides observed in the Multiple Sequence Alignment (MSA) used to construct the model. Mathematically, the emission probability $e_i(x)$ for observing residue x at position i in the alignment is calculated as:

$$e_i(x) = \frac{\text{Count of residue } x \text{ at position } i}{\text{Total count of residues at position } i}.$$

Here, i represents the position in the alignment, and x represents a specific amino acid or nucleotide. The count of residue x at position i is divided by the total count of residues at position i to normalize the probability. Emission probabilities reflect the amino acid composition of each column in the MSA. Unevenly distributed emission probabilities indicate positions that are more conserved, where a specific amino acid or nucleotide is frequently observed across sequences in the alignment. Conversely, evenly distributed emission probabilities suggest positions with more variability, where multiple residues or nucleotides may be present.

Transition Probabilities Transition probabilities in a Profile HMM govern the likelihood of transitioning between different states within the model. These probabilities capture the patterns of insertions, deletions, and matches observed in the MSA. Mathematically, the transition probability t_{ij} from state i to state j is calculated as:

$$t_{ij} = \frac{\text{Count of transitions from state } i \text{ to state } j}{\text{Total count of transitions from state } i}$$

Here, i and j represent different states in the model, such as match, insertion, or deletion states. The count of transitions from state i to state j is divided by the total count of transitions originating from state i to normalize the probability. Transition probabilities reflect the probabilities of insertions, deletions, and matches within the MSA. Higher transition probabilities between match states indicate regions of sequence conservation, where consecutive residues are more likely to align without gaps. Conversely, higher transition probabilities from match states to insertion or deletion states suggest regions with more variability, where insertions or deletions are common. By capturing these probabilities, Profile HMMs effectively model the indel patterns observed in the alignment.

Calculating Sequence Probability Using Profile HMMs Before delving into the computational dynamics of the Viterbi algorithm for sequence alignment using Profile Hidden Markov Models (HMMs), we investigate how to calculate the probability of observing a specific sequence along a particular path through the model.

Sequence and Path Definition Consider a sequence $X = x_1, x_2, \dots, x_L$ and a corresponding path $\pi = \pi_1, \pi_2, \dots, \pi_L$ through a Profile HMM, where each π_i represents the state (Match, Insertion, or Deletion) at position i in the sequence. Each state in the path emits a sequence character (deletion

states, which are silent, emits a token $x_i = \delta$ with a probability $e_{\pi_i}(\delta) = 1$), and the transition from one state to the next is governed by the structure of the HMM.

Probability Calculation For Profile HMMs, just as other HMMs, we assume conditional independence of the emitted sequence given its path through the states. This means that the probability of observing a particular sequence is determined solely by the sequence of states in the HMM, independent of the emissions from other states. The probability of observing the sequence X along the path π through the Profile HMM is then becoming the product of all relevant emission probabilities and transition probabilities along the path. Mathematically, this is expressed as:

$$P(X, \pi) = \prod_{i=1}^L t_{\pi_{i-1}, \pi_i} \cdot e_{\pi_i}(x_i)$$

Here, t_{π_{i-1}, π_i} is the transition probability from the preceding state π_{i-1} into state π_i (with π_0 denoting the start state), and $e_{\pi_i}(x_i)$ represents the emission probability of observing symbol x_i from state π_i . It's important to note that the emission probability of a deletion state is always 1 since deletions do not emit any symbols, and hence emits “no symbol” with a probability of 1.

Step-by-Step Breakdown

1. **Initialization:** Start from the beginning of the sequence and the corresponding initial state in the path. The initial state typically has no preceding state, so the initial transition probability may be a special case, often set based on the model's parameters.
2. **Iteration:** For each position i from 1 to L in the sequence and path:
 - Compute the emission probability $e_{\pi_i}(x_i)$ if π_i is a match or insertion state. For deletion states, this value is 1.
 - Compute the transition probability t_{π_{i-1}, π_i} from the previous state π_{i-1} to the current state π_i .
3. **Multiplication:** Multiply all computed probabilities (emission and transition) to get the total probability of observing the sequence X given the path π .

Sequence Alignment with Profile HMMs using Viterbi Algorithm When we want to score a sequence with a profile HMM, we normally don't have a path available that likely generated a certain sequence. However, it turns out that we can use dynamic programming to find an optimal path. This is done using the Viterbi algorithm, adapted for Profile Hidden Markov Models (Profile HMMs). Here's a concise breakdown of the process:

Overview: Given a Profile HMM and a query sequence, the Viterbi algorithm identifies the most probable alignment between the sequence and the model. This alignment accounts for matches, insertions, and deletions. The algorithm computes a dynamic programming matrix $V(i, j)$, where each entry represents the probability of the most likely path that aligns the first i residues of the sequence to state j of the HMM.

Steps:

1. **Initialization:**
 - Initialize the first column of the Viterbi matrix with probabilities for starting in the appropriate states.
 - For the start state:

$$V(0, 0) = 1.0 \tag{8}$$

meaning that before reading any residues, the probability of being in the start state is 1.

2. **Recursion:**

- Iterate through each residue in the sequence, updating the Viterbi matrix based on transition and emission probabilities.
- Update each cell in the Viterbi matrix using the following formula:
 $V(i, j) = \max_{k \leq j} [V(i-1, k) \cdot t_{kj}] \cdot e_j(x_i)$, when j is a match or insert state.
 $V(i, j) = \max_{k < j} [V(i, k) \cdot t_{kj}]$, when j is a delete state.

3. Termination:

- Once the entire sequence has been processed, identify the highest probability in the last column of the Viterbi matrix. This represents the likelihood of the most probable path through the model.

4. Backtracking:

- Trace back through the Viterbi matrix, starting from the cell with the highest probability in the last column. Record the path through the model that corresponds to this maximum probability.

5. Alignment and Scoring:

- Translate the identified path into an alignment between the query sequence and the Profile HMM. Match states represent aligned residues, insertion states represent residues not accounted for in the model, and deletion states represent gaps in the sequence.
- The Viterbi score for the sequence is simply the probability of this optimal path. This provides a quantitative measure of how well the sequence fits the model.

Using Viterbi for Sequence Classification The Viterbi algorithm is not only useful for aligning a sequence to a single Profile HMM, but also for **scoring new sequences against entire libraries of HMMs**. For example, the Pfam database contains thousands of curated profile HMMs representing protein domains. By scoring a new sequence against all models in Pfam, one can identify which domain families are present and classify the sequence accordingly.

Comparison to MSAs

Advantages Profile HMMs provide a probabilistic framework that captures the variability and evolutionary patterns within multiple sequence alignments (MSAs), allowing for more flexible and robust sequence comparison. Unlike traditional MSAs, Profile HMMs can incorporate gap penalties and position-specific scoring, making them more suitable for handling insertions and deletions. Additionally, Profile HMMs can model the evolutionary history of sequences and predict the likelihood of a sequence being a member of a protein family, which MSAs do not inherently provide.

Disadvantages Despite their strengths, Profile HMMs have limitations. The conditional independence criterion, which assumes that the emitted sequence is independent of other emissions given the hidden state, limits the model's ability to capture correlations between different positions in the sequence. As a result, Profile HMMs cannot model interdependencies between positions within a sequence. I.e. if amino acids covary between two positions in a sequence the HMM will not be able to represent or recognize such a pattern.

Code describing the Viterbi algorithm

The Decoding Problem The previous section described how a profile HMM models a multiple sequence alignment: match states capture conserved positions, insert states allow extra residues, and delete states account for gaps. Given such a model and a new query sequence, the natural question is:

Which path through the HMM states best explains this sequence?

This is called the **decoding problem**. Each path assigns every residue of the query to a specific state: a match state (the residue aligns to a conserved column), an insert state (the residue is an extra insertion), or a delete state (a column of the model is skipped). The path that maximises the product of all transition and emission probabilities along the route is the **Viterbi path**, and the algorithm that finds it efficiently using dynamic programming is the **Viterbi algorithm**.

Concretely, the algorithm fills a matrix $V(s, i)$ where entry (s, i) is the probability of the single most likely path that ends in state s having consumed i residues of the sequence. The recursion is

$$V(s, i) = e_s(x_i) \cdot \max_{s'} [V(s', i') \cdot a_{s' \rightarrow s}] \quad (9)$$

where $e_s(x_i)$ is the emission probability of residue x_i in state s , $a_{s' \rightarrow s}$ is the transition probability from s' to s , and i' equals $i - 1$ for emitting states or i for silent states (deletions). After filling the matrix, backtracking through the recorded choices gives the optimal alignment of the query to the profile.

The code below implements this for a small four-column DNA profile HMM.

We start with defining HMM as a list of states, each state defined as a dictionary. For keeping the example simple we model alignments of DNA sequences i.e. the model emits sequences from an alphabet of 4 letters, rather than the 20 letters of amino acid sequences. We designed the HMM so that the match states favors a sequence “ACGT”. We also include a dictionary giving relative positions to the previous state of each type (M, I, D, etc.)

```
import numpy as np
```

```
profile_hmm = [
    {'type': 'S', 'emission': {}, 'transition': {'M': 0.},
    {'type': 'I', 'emission': {'A': 0.2, 'C': 0.3, 'G': 0.3, 'T': 0.2}, 'transition': {'M': 0.},
    {'type': 'D', 'emission': {}, 'transition': {'M': 0.},
    {'type': 'M', 'emission': {'A': 0.6, 'C': 0.1, 'G': 0.2, 'T': 0.1}, 'transition': {'M': 0.},
    {'type': 'I', 'emission': {'A': 0.2, 'C': 0.3, 'G': 0.3, 'T': 0.2}, 'transition': {'M': 0.},
    {'type': 'D', 'emission': {}, 'transition': {'M': 0.},
    {'type': 'M', 'emission': {'A': 0.2, 'C': 0.6, 'G': 0.1, 'T': 0.1}, 'transition': {'M': 0.},
    {'type': 'I', 'emission': {'A': 0.2, 'C': 0.3, 'G': 0.3, 'T': 0.2}, 'transition': {'M': 0.},
    {'type': 'D', 'emission': {}, 'transition': {'M': 0.},
    {'type': 'M', 'emission': {'A': 0.1, 'C': 0.2, 'G': 0.5, 'T': 0.2}, 'transition': {'M': 0.},
    {'type': 'I', 'emission': {'A': 0.2, 'C': 0.3, 'G': 0.3, 'T': 0.2}, 'transition': {'M': 0.},
    {'type': 'D', 'emission': {}, 'transition': {'E': 1.},
    {'type': 'M', 'emission': {'A': 0.2, 'C': 0.2, 'G': 0.2, 'T': 0.4}, 'transition': {'E': 1.},
    {'type': 'I', 'emission': {'A': 0.2, 'C': 0.3, 'G': 0.3, 'T': 0.2}, 'transition': {'E': 1.},
    {'type': 'E', 'emission': {}, 'transition': {}},
]
```

```
prev_rel_states = { # Relative position to previous state of each type
    'S': {},
    'M': {'S': -3, 'M': -3, 'I': -2, 'D': -4},
    'I': {'S': -1, 'M': -1, 'I': 0, 'D': -1},
    'D': {'S': -2, 'M': -2, 'I': -1, 'D': -3},
}
```

```

    'E': {          'M': -2, 'I': -1, 'D': -3},
}

```

Then we define the dynamic programming algorithm to compute the Viterbi matrix, and back-tracking the optimal path (the Viterbi path) through the model.

```

def viterbi(profile_hmm, sequence):
    num_states = len(profile_hmm)
    num_bases = len(sequence)

    # Initialize the Viterbi and path matrices
    viterbi_matrix = np.zeros((num_states, num_bases+2))
    viterbi_path = np.zeros((num_states, num_bases+2), dtype=int)

    # Initialize the first column of the Viterbi matrix
    viterbi_matrix[0, 0] = 1.0

    # Fill the Viterbi matrix
    for base_idx in range(1, num_bases+2):
        for state in range(num_states):
            transition_probs = {}
            current_type = profile_hmm[state]['type'] # Is this a 'M', 'I', or 'D' state?
            isCurrentSilent = not profile_hmm[state]['emission']
            # Get the previous states that can transition to the current state
            prev_abs_states = { t : state + rel for t, rel in prev_rel_states[current_type].it
            # Get the previous base index, it is different for silent states (S, E and D)
            prev_abs_base = base_idx if (isCurrentSilent) else base_idx -1
            for prev_type, prev_abs_state in prev_abs_states.items():
                transition_prob = profile_hmm[prev_abs_state]['transition'][current_type]
                prev_score = viterbi_matrix[prev_abs_state, prev_abs_base]
                transition_probs[prev_abs_state] = transition_prob * prev_score
            if transition_probs: # Check if the list is not empty
                max_prev_state = max(transition_probs, key=transition_probs.get)
                max_transition_prob = transition_probs[max_prev_state]
                # print(max_prev_state, max_transition_prob)
                if profile_hmm[state]['emission'] and base_idx <= num_bases:
                    emission_prob = profile_hmm[state]['emission'].get(sequence[base_idx -1],
                    else:
                        emission_prob = 1.0
                viterbi_matrix[state, base_idx] = max_transition_prob * emission_prob
                viterbi_path[state, base_idx] = max_prev_state
    # Trace back to find the most probable path
    state = num_states - 1
    base_idx = num_bases + 1
    letter=' -'
    best_path = []
    while base_idx>=1 and state>0:
        best_path.append((state, profile_hmm[state]['type'], letter ))
        state = viterbi_path[state, base_idx]
        isSilent = not profile_hmm[state]['emission']
        if isSilent:
            letter = ' -'
        else:
            base_idx -= 1

```

```

        letter = sequence[base_idx - 1]
        best_path.append((state, profile_hmm[state]['type'], letter ))
        best_path.reverse()
        return best_path

```

Lets first try with a “ACGT” sequence.

```

decoded_path = viterbi(profile_hmm, 'ACGT')
print("Decoded path:")
for state, type, letter in decoded_path:
    print(f"State {state} of type {type} emitted {letter}")

```

```

Decoded path:
State 0 of type S emitted -
State 3 of type M emitted A
State 6 of type M emitted C
State 9 of type M emitted G
State 12 of type M emitted T
State 14 of type E emitted -

```

We could try a sequence that is longer than the model.

```

decoded_path = viterbi(profile_hmm, 'ACAAGT')
print("Decoded path:")
for state, type, letter in decoded_path:
    print(f"State {state} of type {type} emitted {letter}")

```

```

Decoded path:
State 0 of type S emitted -
State 3 of type M emitted A
State 6 of type M emitted C
State 7 of type I emitted A
State 7 of type I emitted A
State 9 of type M emitted G
State 12 of type M emitted T
State 14 of type E emitted -

```

And a shorter sequence.

```

decoded_path = viterbi(profile_hmm, 'AGT')
print("Decoded path:")
for state, type, letter in decoded_path:
    print(f"State {state} of type {type} emitted {letter}")

```

```

Decoded path:
State 0 of type S emitted -
State 3 of type M emitted A
State 5 of type D emitted -
State 9 of type M emitted G
State 12 of type M emitted T
State 14 of type E emitted -

```

0.1.5 Sequence Retrieval

The BLAST Algorithm for Sequence Retrieval

Introduction BLAST (Basic Local Alignment Search Tool) is a powerful algorithm designed for comparing biological sequences, such as protein amino acid sequences or DNA/RNA nucleotide sequences Altschul et al. [1990]. Developed in 1990, BLAST has become one of the most widely used tools in bioinformatics, cited more than a hundred thousand times.

The primary purpose of BLAST is to compare a query sequence against a database of sequences, identifying those with a similarity above a certain threshold. This enables researchers to efficiently find sequences that resemble the query, facilitating further biological research.

Background and Development BLAST was designed to address a fundamental challenge in bioinformatics: how to efficiently search large genomic databases. Its heuristic algorithm offers a much faster alternative to traditional methods like Smith-Waterman and Needleman-Wunsch, which focus on optimal alignment but are computationally intensive.

BLAST's success and widespread adoption stem from its balance of speed and sensitivity, making it an essential tool for biological research. Its development was also supported by a novel stochastic model Karlin and Altschul [1990], the so-called Karlin-Altschul statistics, forming the statistical foundation for the tool.

K-mer indexing

What are k-mers? Before we describe the details of BLAST, we need to describe the concept of k-mers. A k-mer is a contiguous substring of length k derived from a larger sequence:

- **Generation:** K-mers are generated by sliding a window of length k across a sequence, creating overlapping substrings.
- **Examples:** For a DNA sequence “AGCTAGC”, 3-mers (k=3) would include “AGC”, “GCT”, “CTA”, etc.
- **Importance:** K-mers are useful for representing sequences, identifying sequence patterns, and conducting sequence comparisons.

K-mers can be used for indexing sequences, making sequence retrieval more efficient:

- **Hash Table Creation:** A hash table or other data structure can store k-mers as keys, mapping them to sequence locations.
- **Fast Lookups:** By indexing k-mers from database sequences, BLAST can quickly look up similar k-mers from the query sequence, avoiding the need for a full pairwise alignment.
- **Word Hits:** K-mer matches between the query and database sequences, termed “word hits”, serve as initial points of comparison for BLAST.

Dividing “APEPTIDE” into 3-mers

The process of dividing a sequence into k-mers involves taking every possible contiguous subsequence of length k from the given sequence. Here's how it looks for the 3-mers of “APEPTIDE”:

- APEPTIDE
- APEPTIDE
- APEPTIDE
- APEPTIDE
- APEPTIDE
- APEPTIDE

This results in the following 3-mers:

- APE
- PEP
- EPT
- PTI

- TID
- IDE

Indexing Before BLAST can efficiently search a database, it is critical that one index the sequences. Here's how BLAST indexes sequences: BLAST creates a lookup table where each entry corresponds to a k-mer (the default word size is 11 for nucleotides and 3 for proteins). For each k-mer in the database sequences, the algorithm stores its position within the sequence.

The BLAST Algorithm The Basic Local Alignment Search Tool (BLAST) is a heuristic algorithm designed to create local alignments, helping to find the closest homologs of sequences in a database. The algorithm leverages indices of k-tuples from the sequence database, with k typically ranging from 3 to 6 for proteins and 6 to 20 for DNA. BLAST follows these key steps:

1. **Word Hits Generation:** The query sequence is divided into k-tuples, which are compared to pre-computed k-tuples from the database sequences.
2. **Seeding:** The matching k-tuples, known as “seeds”, are used as anchor points for the alignment procedure. The list of k-tuples is expanded to include any k-tuples that match with an alignment score above a threshold T .
3. **Extension:** BLAST extends these seeds in both directions, creating larger alignments. This extension continues until the alignment score drops below a threshold, preventing poor alignments from extending. These extended matches are known as high-scoring segment pairs (HSPs).
4. **Alignment Scoring:** Each extended alignment is scored based on the similarity between the aligned regions, taking into account matches, mismatches, and gaps.
5. **Statistical Evaluation:** BLAST calculates the statistical significance of each alignment using E-values, indicating how many alignments of the same quality would be found by chance in a database of a given size.
6. **Output:** The algorithm then lists all alignments above a threshold score, providing details such as alignment positions, scores, and statistical significance.

This combined approach gives an overview of BLAST, highlighting its key features and processes.

E-Values and Their Meaning BLAST assesses the statistical significance of alignments using the Gumbel extreme value distribution (EVD) model. This model estimates the probability of an alignment score being due to chance. An E-value (expect value) is a statistical measure used in BLAST to assess the significance of an alignment with a score S .

- **Definition:** The E-value represents the number of alignments with a score equal to or greater than S that would be expected to occur by chance in the searched database.
- **Interpretation:** Lower E-values indicate more significant alignments. For example, an E-value of 0.01 means that, on average, 0.01 alignments of this quality or better would be expected by chance for a query sequence of the same length searched against a database of the same size — i.e. roughly one such chance alignment per hundred searches. (Strictly, the E-value is an expected *number* of hits rather than a probability, but for small E-values it closely approximates the probability of seeing at least one such alignment by chance.)

The E-values in BLAST searches provide a direct indication of alignment quality:

- **Significance Threshold:** Alignments with E-values below a certain threshold (e.g., 0.05) are considered significant, indicating a meaningful biological relationship.
- **Comparison:** E-values allow for comparing the statistical significance of different alignments, making them a valuable metric for researchers.
- **Database Size Influence:** The size of the database affects E-values, as larger databases increase the likelihood of chance alignments, resulting in higher E-values.

How is the BLAST E-value Calculated? The E-value (expect value) is a statistical measure used in BLAST to assess the significance of an alignment between a query sequence and database sequences. It reflects the expected number of alignments that would occur by chance in a given database search. The E-value is calculated based on the alignment score S , the search space size $m \times n$, and parameters derived from the scoring system and database composition, such as the Karlin-Altschul parameters (K and λ). The formula for the E-value is:

$$E = K m n e^{-\lambda S}$$

Where:

- m is the length of the query sequence.
- n is the length of the database, which is the sum of the lengths of all the sequences in the database.
- K and λ are the Karlin-Altschul parameters. They can be estimated from large sets of random sequence alignments:
 - λ normalizes the alignment score.
 - K scales the E-value based on the database and sequence lengths.
- S is the alignment score, calculated from the selected scoring matrix and the alignment of residues. It accounts for the sum of substitution and gap scores for the aligned residues.

The E-value is directly proportional to the search space size ($m \times n$) and inversely proportional to the exponential function of the alignment score (S). Consequently, larger databases provide more opportunities for chance alignments, resulting in higher E-values (indicating weaker statistical significance) for the same level of sequence similarity.

How Are K and λ Obtained? When BLAST was first developed, the parameters K and λ were estimated by aligning large numbers of randomized sequences with the same background composition as the database. The distribution of alignment scores from these random comparisons follows an extreme value distribution (EVD), as predicted by Karlin-Altschul theory. By fitting the empirical score distribution to the EVD, appropriate values of K and λ were obtained for each scoring matrix and residue composition.

Formally, λ is defined as the unique positive solution to the equation

$$\sum_{i,j} p_i q_j e^{\lambda s_{ij}} = 1, \quad (10)$$

where s_{ij} is the substitution score for aligning residues i and j , and p_i , q_j are their background frequencies. Once λ is determined, K is fitted so that the theoretical distribution

$$P(S \geq x) \approx 1 - \exp(-K m n e^{-\lambda x}) \quad (11)$$

matches the observed distribution of random alignment scores.

Today, BLAST does not generate random sequences for each search. Instead, it uses lookup tables of pre-computed values for K and λ , derived from these earlier large-scale calibrations. For standard scoring matrices (e.g. BLOSUM62, PAM250) and typical residue frequencies, these values are stable and can be reused. In addition, BLAST may apply composition-based adjustments if the query or database composition deviates significantly from the assumptions used in the pre-computed tables.

Types of BLAST BLAST comes in a couple of different versions, depending on its usage. Here are the main versions of BLAST:

- **BLASTn (Nucleotide BLAST):** Compares nucleotide sequences against a database or another sequence to identify evolutionary relationships. Useful in phylogenetics studies.
- **tBLASTn:** Searches for proteins in untranslated DNA sequences. Takes a protein sequence and compares it to all potential translations of a nucleotide sequence. Useful for finding protein-coding regions in ESTs and HTGs.

- **BLASTx:** Compares a nucleotide query, translated into six possible protein sequences, against a protein database. This tool is essential when DNA sequence reading frames are uncertain or contain errors that could impact protein coding. It provides combined statistics across all frames for new DNA analyses.
- **BLASTp (Protein BLAST):** Used for comparing protein sequences against a database (e.g., nr database). It helps in identifying proteins by finding similar sequences in known protein databases.

The FASTA Format BLAST is using FASTA format as input format for its databases and queries. The FASTA format is a widely adopted standard for representing nucleotide and protein sequences in bioinformatics. Developed in the 1980s for the FASTA sequence alignment software, it has since become a versatile and essential format for storing and sharing sequence data. The FASTA format consists of two key components:

1. **Header Line:** Each sequence begins with a header line, which starts with a greater-than symbol (>). The text following this symbol provides a description of the sequence. This description often includes information such as a unique identifier (e.g., accession number), the source organism, and other metadata. For example:

```
>sp|P12345|PROT_HUMAN Human Protein Name [Homo sapiens]
```

In this example, “sp” indicates the Swiss-Prot database, “P12345” is the accession number, “PROT_HUMAN” is the unique identifier, and additional information follows.

2. **Sequence Lines:** Below the header line, the sequence data is written as plain text. For nucleotide sequences, this consists of a string of the bases A, C, G, and T (or U for RNA). For protein sequences, it consists of a string of the standard 20 amino acid single-letter codes. The sequence can be broken into multiple lines, making it easier to read and process:

```
ATGCGTACGTGACGT
CGTGAGCTAGTCAGT
```

These sequence lines represent the data to be analyzed and compared.
Combining the two you get a fasta record e.g.

```
>sp|C9JLW8|MCRI1_HUMAN Mapk -regulated corepressor -interacting protein 1 OS=Homo sapiens OX=9
MTSSPVSRRVYNGKRTSSPRSPSSSEIFTPAHEENVRFIYEAWQGVERDLRGQVPGGER
GLVEEYVEKVPNPSLKTFFKPIDLSCLKRRSTQDAKKS
```

Usage and Applications The FASTA format’s simplicity and clarity make it ideal for storing and sharing sequence data. It is supported by most bioinformatics tools, including alignment algorithms, database search tools, and genome browsers. Additionally, the format is highly adaptable, allowing for easy conversion to and from other sequence formats.

FASTA files can contain multiple sequences, each represented by its own header and sequence lines, making them an efficient way to store large datasets. They are used in bioinformatics pipelines, providing a way to manage and share sequence information. Note, however, that FASTA is not a well-defined format, and there are multiple variants in how both headers and sequence lines should be formatted.

You can for instance get all the canonical amino acid sequences of the human genome as a single FASTA file

References

Code for BLAST-styled queries

Here is some code demonstrating some of the basic building blocks of the BLAST algorithm: the database and query indexing, some basic joining of matches, and the display of the results.

Note that the code just tries to give the outline of some of the aspects of the algorithm, and the real BLAST contains code for extending matches, calculating scores and extensions etc. which is not part of this notebook.

build_index function: This function creates an index from a given sequence for subsequences (k-mers) of length k. Each k-mer is mapped to all positions where it appears in the sequence, here given as a tuple of the sequence number and its position within the sequence. This step of the processing could be precalculated for each database prior to encountering any query.

```
def build_index(sequences, k):
    """Builds indexes from multiple sequences for subsequences of length k."""
    index = {}
    for seq_id, sequence in enumerate(sequences):
        # seq_id will be a number, 0 for first sequence, 1 for second, etc
        for i in range(len(sequence) - k + 1):
            k_mer = sequence[i:i+k]
            if k_mer in index:
                index[k_mer].append((seq_id,i))
            else:
                index[k_mer] = [(seq_id,i)]
    return index
```

query_index function: This function takes a query sequence and finds all k-mers in the query that are present in the index, along with their positions in the original sequence.

```
def query_index(query, index, k):
    """Queries multiple indexes and finds positions where the query's k -mers match the k -mer
    matches = {}
    for pos_in_query in range(len(query) - k + 1):
        k_mer = query[pos_in_query:pos_in_query+k]
        if k_mer in index:
            # we found a match. Store the position_in_query
            matches[k_mer] = pos_in_query
    return matches
```

The **merge_matches** function consolidates all matching k-mers from the query into continuous sequences for each sequence in the database. It first rearranges the matches by sequence index and position, then sorts and merges the k-mers into longer, contiguous segments that represent potential alignments between the query and the database sequences.

```
def merge_matches(matches, query, index, k):
    """Merges all k -mer matches into continuous sequences for each database sequence."""
    # First rearrange the matches so that they are grouped by their matched database sequence
    rearranged_matches = {}
    for k_mer, pos_query in matches.items():
        index_matches = index[k_mer]
        for seqid, pos_in_seq in index_matches:
            if seqid not in rearranged_matches:
                rearranged_matches[seqid] = []
            rearranged_matches[seqid].append((pos_query, pos_in_seq, k_mer))
    matches = []
```

```

# Walk through each matched sequence and arrange all matched k -tuples
for seqix, list_of_pos2_kmer in rearranged_matches.items():
    list_of_pos2_kmer.sort() # sort by query position
    match = " -" * len(query)
    previous_pos_i, start_of_match = -1, None
    for pos_q, pos_i, k_mer in list_of_pos2_kmer:
        if not start_of_match:
            start_of_match = pos_i - pos_q
        # Check that the match does not come after the current match
        if previous_pos_i < pos_i:
            match = match[:pos_q] + k_mer + match[pos_q+k:]
            previous_pos_i = pos_i
        matches.append((match, seqix, start_of_match))
    return matches

```

Now we have all the code we need for running a test run of our code.

```

# Example sequences in the database
sequences = ["APEPTIDE", "PEPTIDEA", "DIFFERENT", "TIDEAPEP", "REPTILE"]

# Query sequence
query = "PEPTID"

# Build index for a k -mer length of 3 from multiple sequences
k = 3
index = build_index(sequences, k)
#print("Indexes:", index)

# Query the indexes
matches = query_index(query, index, k)
#print("Matches:", matches)

# Merge matches into continuous sequences
merged_alignments = merge_matches(matches, query, index, k)

print("Alignments:")
for seq_match, seq_ix, start_of_match in merged_alignments:
    print("DB:  " + sequences[seq_ix] )
    print("MA:  " + ' -'*start_of_match + seq_match + ' -'*(len(sequences[seq_ix]) -start_of_m
    print("QU:  " + ' '*start_of_match + query)
    print()

Alignments:
DB:  APEPTIDE
MA:  -PEPTID -
QU:  PEPTID

DB:  PEPTIDEA
MA:  PEPTID - -
QU:  PEPTID

DB:  TIDEAPEP
MA:  - - - - -PEP - - -
QU:  PEPTID

```

DB: REPTILE
MA: -EPTI - -
QU: PEPTID

The algorithm finds matches to the first, second, fourth and fifth sequences

0.2 Phylogenetics

0.2.1 Introduction to Phylogenetics

Introduction to Phylogenetics

Phylogenetics uses heritable differences among organisms or genes to infer their evolutionary history. A phylogenetic tree is therefore a hypothesis based on present-day observations, not a direct record of the past.

Molecular evolution and speciation Mutations continually introduce new variants into a population. Natural selection changes the frequencies of variants that affect reproductive success, while **genetic drift** changes frequencies by chance, especially in small populations. A new variant may disappear or become **fixed**, meaning that it eventually occurs in every member of the population.

When gene flow between two populations is reduced, mutations arise and become fixed independently in each population. The populations consequently accumulate different substitutions. Selection can promote divergence when the populations experience different environments, while drift can produce divergence even without an adaptive difference. If reproductive isolation develops, the populations form separate species.

Descendant species inherit most of their DNA from their common ancestral population. Shared and differing characters in modern sequences therefore contain an incomplete signal of their shared history. Phylogenetic methods use this signal to infer where evolutionary lineages diverged.

Important

A mutation is a change in a DNA molecule. A substitution is a mutation that has become fixed when comparing evolutionary lineages. Phylogenetic models usually describe substitutions, although the two terms are sometimes used loosely.

Phylogenies as inferred evolutionary histories A **phylogeny** is the evolutionary history of a group of replicating entities, such as species, viruses or genes. We normally observe only sequences from contemporary taxa. Common ancestors, ancestral sequences and divergence events are inferred from those observations.

Many histories could potentially explain the same data. A tree-building method uses explicit criteria—such as minimum evolutionary change or maximum likelihood—to select or compare hypotheses. The resulting tree should not be treated as certainty: its interpretation depends on the data, evolutionary model and statistical support.

Parts of a tree

- **Leaves** or **terminal nodes** represent the sampled taxa or genes.
- **Internal nodes** represent inferred divergence events.
- A **branch** connects two nodes and represents an evolving lineage.
- The **root**, when known, represents the common ancestor of all observations and gives the direction of time.
- Two descendants sharing an immediate ancestor are often called **sister groups**.

An internal node is not one of the sampled descendants. For example, if A is sister to B, neither A nor B is the ancestor of the other.

Topology and branch lengths The **topology** is the branching order. A and B are more closely related to each other than either is to C only when A and B share a more recent common ancestor.

Branch lengths may contain additional information. In a **phylogram**, they are usually proportional to evolutionary change, commonly measured in expected substitutions per site. In a **cladogram**, only the topology is informative. Horizontal or vertical spacing chosen merely to make a tree readable has no biological meaning.

Equivalent drawings and rotations Rotating the descendants around an internal node does not change a topology. Thus $((A,B), (C,D))$ describes the same relationships as $((B,A), (D,C))$. Compare trees by their groups of descendants, not by the left-to-right order of their leaves.

Newick format Newick is a compact, computer-readable representation of trees. Parentheses enclose descendants sharing a node, commas separate branches, and a semicolon ends the tree:

```
((A,B),(C,D));
```

Branch lengths follow a colon:

```
((A:0.10,B:0.10):0.08,(C:0.05,D:0.05):0.13);
```

Internal node labels, including support values, may appear after a closing parenthesis.

Clades A **clade** contains an ancestor and all of its descendants and is therefore monophyletic. A group assembled from several lineages while excluding their most recent common ancestor, or some of its descendants, is not a clade. Such groupings may be polyphyletic or paraphyletic.

Exercises

1. Explain why rotating branches around an internal node does not create a new tree.
2. In `((A,B),(C,D));`, which pairs are sister taxa? Is A more closely related to C than to D?
3. Write a Newick tree in which A and C are sister taxa and B is the outgroup.
4. Why is an inferred internal node not normally interpreted as one of the sampled sequences?

From Sequences to Phylogenetic Data

A defensible phylogeny begins before a tree-building program is run. The usual workflow is to collect homologous sequences, align homologous characters, infer a tree and evaluate its reliability.

Homology Characters compared in a phylogenetic analysis must be related by descent. Sequences sharing ancestry are **homologous**; homology is a relationship, not a percentage of similarity.

- **Orthologs** diverged through speciation and often reflect the species history.
- **Paralogs** diverged through gene duplication and reflect the history of gene copies.
- **Xenologs** have a history involving horizontal gene transfer.

If a paralog is mistaken for an ortholog, the inferred gene tree may disagree with the species tree even when the tree inference itself is correct.

Gene trees and species trees A sequence alignment normally informs a **gene tree**. It need not equal the species tree. Duplication, gene loss, horizontal transfer, recombination and incomplete lineage sorting can all produce genuine differences. Multiple carefully selected genes are therefore often used to infer organismal relationships.

Horizontal gene transfer is particularly important in prokaryotes. Conserved housekeeping genes are frequently selected because many are transferred less often than genes involved in specialized functions, although no marker is universally immune.

Choosing sequence data The marker must vary at a suitable rate for the evolutionary timescale.

- Fast-evolving nucleotide regions can resolve recent divergences but may saturate over long times.
- Protein sequences retain useful signal across deeper divergences because several nucleotide changes can encode the same amino acid.
- Structural RNAs contain conserved regions useful for comparing distant organisms.
- Several genes can be concatenated to increase the number of characters, provided their histories and evolutionary properties are sufficiently compatible.

More data do not automatically remove systematic bias. Taxon sampling, contamination, sequence quality and model suitability also matter.

Alignment defines the characters Each column in a multiple sequence alignment is treated as a set of homologous character states inherited from a common ancestral position. An incorrect alignment therefore creates false substitutions and can strongly support a false tree.

Before tree inference, inspect the alignment for uncertain regions, excessive gaps, non-homologous sequence ends, frame disruptions and potential recombination. Ambiguous regions may need to be realigned, masked or removed, but such decisions should be documented.

A practical checklist

1. State whether the target is a gene tree or species tree.
2. Verify sequence identity and direction.
3. Distinguish orthologs from paralogs where possible.
4. Select markers appropriate to the divergence timescale.
5. Construct and inspect the multiple alignment.
6. Consider recombination or horizontal transfer.
7. Record all filtering and alignment decisions.

0.2.2 Distance-Based Phylogenetics

Evolutionary Distances

Distance-based methods summarize each pair of aligned sequences by a single number. The simplest number is the observed fraction of differences, but evolutionary models are needed when multiple changes can occur at the same site.

The p-distance For two aligned sequences, let L be the number of compared positions and m the number of mismatches. Positions containing gaps are commonly excluded. The **p-distance** is

$$p = \frac{m}{L}. \quad (12)$$

It is easy to interpret: if 8 of 100 compared sites differ, $p = 0.08$. However, it measures observed differences rather than the number of evolutionary changes.

Multiple substitutions and saturation A site may change several times. A later substitution can overwrite an earlier one or restore the original state. Consequently, two identical bases need not have remained unchanged, and one observed mismatch can represent several substitutions.

As divergence increases, new substitutions increasingly strike previously changed sites. The observed p-distance then grows more slowly than the true number of substitutions. Eventually it approaches a ceiling—**substitution saturation**—and contains little information about additional evolutionary time.

Jukes-Cantor correction The Jukes-Cantor model assumes equal base frequencies and equal rates for all nucleotide substitutions. Under these assumptions, the corrected distance is

$$d_{JC} = -\frac{3}{4} \ln \left(1 - \frac{4p}{3} \right). \quad (13)$$

The result is an estimate of substitutions per site. For small p , $d_{JC} \approx p$; the correction becomes larger as divergence increases. It is undefined when $p \geq 0.75$, reflecting saturation under the model.

Kimura two-parameter correction Transitions exchange two purines ($A \leftrightarrow G$) or two pyrimidines ($C \leftrightarrow T$). Transversions exchange a purine and a pyrimidine. Transitions are often more frequent, so the Kimura two-parameter model distinguishes them.

Let P be the fraction of compared sites with transitions and Q the fraction with transversions. Then

$$d_{K2P} = -\frac{1}{2} \ln(1 - 2P - Q) - \frac{1}{4} \ln(1 - 2Q). \quad (14)$$

Like every correction, it depends on its assumptions and fails for sufficiently saturated data.

Molecular clocks The **molecular clock** hypothesis states that substitutions accumulate at an approximately constant rate. If each of two lineages evolves at rate r substitutions per site per unit time after splitting t time units ago, their expected distance is approximately

$$d = 2rt. \quad (15)$$

Thus sequence distance can provide information about divergence time when a rate is known. The rate can be calibrated using fossils, dated geological events, ancient DNA or samples collected at known times.

A mutation rate is the rate at which new mutations arise; a substitution rate is the rate at which changes become fixed. They are related but not generally identical. Rates can also differ among genes, sites and lineages because of generation time, selection, population size and molecular constraints. A strict clock is therefore an assumption to test, not a universal law. Relaxed-clock models allow rates to vary.

From distances to a matrix Calculating a distance for every pair of sequences produces a symmetric matrix with zeroes on its diagonal. UPGMA and Neighbor Joining use such a matrix, but they interpret it under different assumptions.

Exercises

1. Two sequences differ at 12 of 100 ungapped positions. Calculate their p-distance.
2. Explain why p-distance increasingly underestimates evolutionary change as sequences diverge.
3. Why might two genes from the same species pair imply different molecular-clock dates?
4. What biological assumptions distinguish Jukes-Cantor from Kimura's two-parameter model?

Code: Evolutionary Distances

In this notebook we calculate distances from an existing multiple sequence alignment. We compare observed p-distances with Jukes–Cantor and Kimura two-parameter corrections, examine substitution saturation, and connect distance to time under a molecular clock. Tree construction is deliberately left for the computer lab.

```
import math
import numpy as np
import matplotlib.pyplot as plt

alignment = {
    'Seq1': 'AAATCC -TATT',
    'Seq2': 'AAAACCGTAAT',
    'Seq3': 'TAGTCCGTAAT',
    'Seq4': 'TAGTCCGTCAT',
}

for name, sequence in alignment.items():
    print(f'{name}: {sequence}')
```

Count comparable positions and substitutions We exclude a column for a pair whenever either sequence contains a gap. A transition is $A \leftrightarrow G$ or $C \leftrightarrow T$; all other nucleotide mismatches are transversions.

```
transition_pairs = {frozenset(('A', 'G')), frozenset(('C', 'T'))}

def substitution_counts(seq_a, seq_b):
    if len(seq_a) != len(seq_b):
        raise ValueError('Sequences must already be aligned')
    compared = transitions = transversions = 0
    for a, b in zip(seq_a.upper(), seq_b.upper()):
        if '-' in (a, b):
            continue
        compared += 1
        if a != b:
            if frozenset((a, b)) in transition_pairs:
                transitions += 1
            else:
                transversions += 1
    return compared, transitions, transversions

L, transitions, transversions = substitution_counts(alignment['Seq1'], alignment['Seq2'])
print(f'Compared sites: {L}')
print(f'Transitions: {transitions}; transversions: {transversions}')
print(f'p -distance: {(transitions + transversions) / L:.3f}')
```

Three nucleotide distances The corrected distances can be undefined for strongly diverged sequences. Returning nan makes this limitation visible instead of allowing an invalid logarithm.

```
def nucleotide_distances(seq_a, seq_b):
    L, ts, tv = substitution_counts(seq_a, seq_b)
    if L == 0:
        return np.nan, np.nan, np.nan
```

```

p = (ts + tv) / L
P, Q = ts / L, tv / L
jc_term = 1 - 4 * p / 3
d_jc = -0.75 * math.log(jc_term) if jc_term > 0 else np.nan
k2p_a, k2p_b = 1 - 2 * P - Q, 1 - 2 * Q
d_k2p = ( -0.5 * math.log(k2p_a) - 0.25 * math.log(k2p_b)
          if k2p_a > 0 and k2p_b > 0 else np.nan)
return p, d_jc, d_k2p

```

```

p, jc, k2p = nucleotide_distances(alignment['Seq1'], alignment['Seq3'])
print(f'p = {p:.3f}, Jukes -Cantor = {jc:.3f}, Kimura 2P = {k2p:.3f}')

```

Construct pairwise distance matrices A distance matrix is symmetric and has zeroes on its diagonal. Notice that corrected distances are at least as large as the observed p-distances for these sequences.

```

def distance_matrices(sequences):
    names = list(sequences)
    matrices = [np.zeros((len(names), len(names))) for _ in range(3)]
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            values = nucleotide_distances(sequences[names[i]], sequences[names[j]])
            for matrix, value in zip(matrices, values):
                matrix[i, j] = matrix[j, i] = value
    return names, matrices

names, (p_matrix, jc_matrix, k2p_matrix) = distance_matrices(alignment)
for label, matrix in [('p -distance', p_matrix), ('Jukes -Cantor', jc_matrix), ('Kimura 2P', k2p_matrix)]:
    print(f'\n{label}')
    print('      ' + ''.join(f'{name:>8}' for name in names))
    for name, row in zip(names, matrix):
        print(f'{name:<7}' + ''.join(f'{value:8.3f}' for value in row))

```

Visualize substitution saturation Under the Jukes–Cantor model, the expected observed difference approaches 0.75 even while the evolutionary distance continues to increase. Multiple substitutions have erased information about earlier changes.

```

true_distance = np.linspace(0, 3, 300)
expected_p = 0.75 * (1 - np.exp( -4 * true_distance / 3))

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(true_distance, expected_p, label='Expected p -distance')
ax.plot(true_distance, true_distance, ' - -', color='grey', label='No multiple hits')
ax.axhline(0.75, color='black', linestyle=':', label='Saturation limit')
ax.set(xlabel='Actual substitutions per site', ylabel='Observed fraction different',
       title='Substitution saturation under Jukes-Cantor', ylim=(0, 1))
ax.legend();

```

Distance and time under a molecular clock For two descendant lineages evolving at the same rate, $d = 2rt$. The factor two appears because substitutions accumulate independently along both branches.

```

def divergence_time(distance, rate):
    return distance / (2 * rate)

```

```
distance = 0.12          # substitutions per site between two sequences
rate = 1e -8             # substitutions per site per year along each lineage
time_years = divergence_time(distance, rate)
print(f'Estimated divergence time: {time_years / 1e6:.1f} million years')

rates = np.array([0.5e -8, 1e -8, 2e -8])
for r in rates:
    print(f'r = {r:.1e}: t = {divergence_time(distance, r) / 1e6:.1f} Myr')
```

Exercises

1. Add a fifth aligned sequence and inspect how the three distance matrices change.
2. Construct a pair for which the p-distance is unchanged but the Kimura distance differs because transitions and transversions occur in different proportions.
3. Increase the observed distance in the molecular-clock example. Which assumptions must remain valid for the time estimate to be meaningful?
4. Why do the Jukes–Cantor and Kimura functions return `nan` for some highly divergent sequence pairs?

UPGMA

UPGMA (Unweighted Pair Group Method with Arithmetic Mean) constructs a rooted tree by repeatedly joining the closest clusters. It is simple and useful for understanding distance-based inference, but its biological assumptions are restrictive.

Agglomerative clustering Initially, each sequence forms a cluster. UPGMA repeats four steps:

1. Find the two clusters with the smallest distance.
2. Join them at a new internal node.
3. Place that node at half their distance.
4. Replace the two clusters by their union and update its distances to all other clusters.

For clusters A and B , UPGMA defines their distance as the arithmetic mean of all pairwise distances between their members:

$$d(A, B) = \frac{1}{|A||B|} \sum_{i \in A} \sum_{j \in B} d(i, j). \quad (16)$$

The word *unweighted* means that every original sequence contributes equally; clusters are therefore weighted by how many sequences they contain.

Worked example Consider this distance matrix:

	S1	S2	S3	S4
S1	0.0	0.2	0.3	0.4
S2	0.2	0.0	0.3	0.4
S3	0.3	0.3	0.0	0.1
S4	0.4	0.4	0.1	0.0

S3 and S4 join first at height $0.1/2 = 0.05$. Their average distances to S1 and S2 are both $(0.3 + 0.4)/2 = 0.35$.

S1 and S2 then join at height $0.2/2 = 0.10$. The distance between the two two-member clusters is

$$\frac{0.3 + 0.4 + 0.3 + 0.4}{4} = 0.35. \quad (17)$$

They join at height $0.35/2 = 0.175$. The branches from the two internal nodes have lengths $0.175 - 0.10 = 0.075$ and $0.175 - 0.05 = 0.125$. The result is

$((S1:0.10, S2:0.10):0.075, (S3:0.05, S4:0.05):0.125);$

Ultrametric assumption UPGMA produces an **ultrametric** tree: every leaf is equally distant from the root. Biologically, this corresponds to a strict molecular clock in which all sampled lineages have accumulated substitutions at the same rate.

If one lineage evolves much faster, UPGMA can cluster by similar observed distance rather than true ancestry and infer the wrong topology. Neighbor Joining was developed to avoid this strict-clock requirement.

Strengths and limitations UPGMA is fast, deterministic and easy to inspect. It is appropriate when distances are approximately ultrametric, and it is widely useful as a clustering method. For general phylogenetic inference, however, its equal-rate assumption must be justified.

Neighbor Joining

Neighbor Joining (NJ) is a distance-based method that constructs an unrooted tree without assuming that all lineages evolve at the same rate.

Why not simply join the closest pair? Two taxa can have a small observed distance because both are close to the center of the tree, not because they are true neighbors. NJ corrects for this by considering each taxon’s total distance from all other taxa.

For n current clusters, define

$$r_i = \sum_k d(i, k)$$

(18)

and the joining criterion

$$Q(i, j) = (n - 2)d(i, j) - r_i - r_j.$$

(19)

The pair with the smallest Q value is joined. This favors pairs that are close to each other relative to their average divergence from the remaining taxa.

Algorithm

1. Begin with one cluster per sequence and a distance matrix.

2. Calculate r_i and the Q matrix.

3. Join the pair with the smallest $Q(i, j)$.

4. Estimate the two branch lengths to their new node u .

5. Update distances using

$$d(u, k) = \frac{d(i, k) + d(j, k) - d(i, j)}{2}.$$

(20)

6. Repeat until the tree is complete.

For a selected pair i, j , the branch from i to u is

$$\delta(i, u) = \frac{1}{2}d(i, j) + \frac{r_i - r_j}{2(n - 2)},$$

(21)

and $\delta(j, u) = d(i, j) - \delta(i, u)$.

UPGMA and NJ compared

Property	UPGMA	Neighbor J
Output	Rooted	Unrooted
Strict molecular clock	Required	Not require
Joining rule	Smallest cluster distance	Smallest co
Leaf-to-root distances	Equal	May differ
Main advantage	Simplicity	Handles ra lineages

NJ is fast and often provides a useful starting tree for more computationally intensive methods. It still compresses each sequence pair into one distance and thus discards information about individual alignment columns.

0.2.3 Character-Based Phylogenetics

Maximum Parsimony

Maximum Parsimony is a character-based method: it evaluates the nucleotides or amino acids in individual alignment columns rather than reducing each sequence pair to one distance.

The parsimony criterion For a proposed topology, ancestral character states are assigned so that each alignment column requires as few changes as possible. The scores of all columns are added. A **most-parsimonious tree** has the smallest total number of changes.

This applies Occam's razor: prefer the history that explains the observations with the fewest evolutionary events. It does not claim that evolution always follows the shortest possible path.

A four-taxon example For four taxa there are three possible unrooted binary topologies:

$((A,B),(C,D))$ $((A,C),(B,D))$ $((A,D),(B,C))$

Suppose one alignment column is:

Taxon	A	B	C	D
State	G	G	T	T

The first topology explains the column with one change between the two groups. Each alternative requires at least two. Constant columns contribute no changes, and some variable columns do not distinguish among topologies. **Parsimony-informative** columns contain at least two states, each represented in at least two taxa.

Searching tree space The number of possible trees grows extremely rapidly with the number of taxa. Exhaustive evaluation is therefore feasible only for small datasets. Practical programs use heuristic searches: begin from one or more trees, rearrange branches, and retain changes that improve the score. A heuristic search may miss the global optimum, and several trees may tie for the best score.

Strengths and limitations Parsimony is intuitive, uses site-specific information and does not require an explicit probabilistic substitution model. However, it treats observed changes as a proxy for actual changes. Multiple substitutions can therefore cause underestimation.

A serious failure mode is **long-branch attraction**: rapidly evolving lineages may independently acquire similar states and be grouped together even though they are not closest relatives. Better taxon sampling and model-based methods can reduce this problem.

Maximum Likelihood

Maximum Likelihood (ML) asks which tree and model parameters make the observed alignment most probable.

Likelihood is conditional on a model For tree T , model parameters θ and sequence data D , the likelihood is

$$L(T, \theta) = P(D \mid T, \theta). \quad (22)$$

It is the probability of the data given the tree—not the probability that the tree is correct. Substitution-model parameters can describe base frequencies, transition and transversion rates, and rate variation among sites. Branch lengths describe expected substitutions per site.

ML selects the topology and parameter values that maximize L . In practice, programs maximize the log-likelihood because products of many small probabilities become sums:

$$\log L_{total} = \sum_s \log L_s, \quad (23)$$

where L_s is the likelihood of alignment site s under the usual assumption that sites evolve independently.

Unknown ancestral states Internal-node states are not observed. Rather than choosing a single reconstruction, likelihood calculations sum over every possible state. For a DNA node, the contribution is summed over A, C, G and T, weighted by equilibrium frequencies and transition probabilities along the branches.

This calculation can be performed efficiently with the pruning algorithm: likelihoods are computed from the leaves toward an internal reference point and reused instead of enumerating every complete ancestral assignment.

Comparing trees For each candidate topology, branch lengths and other model parameters are optimized. The optimized likelihoods are then compared. Calculating one tree’s likelihood is tractable, but the number of possible topologies grows super-exponentially, so large analyses rely on heuristic searches through tree space.

Strengths and limitations ML can model multiple substitutions, unequal base frequencies and rate variation. It is statistically well founded and often more accurate than simple distance or parsimony approaches when the model is suitable. Its results can nevertheless be biased by poor alignment, inappropriate models, insufficient taxon sampling or failure of the heuristic search. It is also computationally demanding.

Bayesian Phylogenetic Inference

Bayesian phylogenetics estimates a posterior distribution over trees and model parameters:

$$P(T, \theta \mid D) \propto P(D \mid T, \theta)P(T, \theta). \quad (24)$$

The likelihood $P(D \mid T, \theta)$ is combined with **prior distributions** on topologies, branch lengths and evolutionary-model parameters. The result describes their relative support after observing the data.

Because the number of trees is enormous, Markov chain Monte Carlo (**MCMC**) is used to sample trees and parameters approximately in proportion to their posterior probabilities. An initial **burn-in** is discarded, and the remaining samples are summarized as a consensus tree. The fraction of retained trees containing a clade is its posterior probability.

Posterior clade probabilities and bootstrap support are not interchangeable. A posterior probability follows from a specified model and priors; bootstrap support measures how consistently a clade is recovered after resampling alignment columns. Posterior probabilities are often numerically higher.

Bayesian results are trustworthy only when the chains have converged and explored the relevant tree space. Independent runs, trace inspection and effective sample sizes are therefore essential diagnostics.

0.2.4 Interpreting Phylogenetic Trees

Rooting Trees and Evaluating Support

Tree inference and tree evaluation answer different questions. An optimality criterion selects a preferred tree, while rooting gives it a direction through time and support analysis assesses which parts are stable under perturbation of the data.

Rooted and unrooted trees An unrooted tree describes splits among taxa but does not identify their common ancestor or the direction of evolution. Neighbor Joining, Maximum Parsimony and Maximum Likelihood commonly produce unrooted trees. Placing the root on different branches can imply different sequences of divergence without changing the unrooted relationships.

Warning

A taxon drawn at the left or bottom of an unrooted tree is not thereby ancestral or early-diverging. Those claims require a root.

Outgroup rooting An **outgroup** is known independently to have diverged before the taxa of primary interest, the **ingroup**. The root is placed on the branch connecting the outgroup to the ingroup.

The outgroup must be outside the ingroup but sufficiently related for trustworthy alignment and phylogenetic signal. A very distant outgroup can introduce alignment ambiguity, saturation and long-branch attraction. Whenever possible, use several appropriate outgroups and examine whether the root is stable.

Other rooting approaches include midpoint rooting and molecular-clock models. Midpoint rooting places the root halfway along the longest leaf-to-leaf path and assumes approximately clock-like evolution.

Bootstrap resampling The phylogenetic bootstrap examines sensitivity to the sampled alignment columns:

1. Sample L columns with replacement from an alignment of length L .
2. Infer a tree from this pseudo-replicate alignment.
3. Repeat the procedure many times, often hundreds or thousands.
4. For each clade in the reported tree, calculate the fraction of replicate trees containing the same split.

Because sampling is with replacement, a replicate contains some original columns several times and omits others.

Interpreting support A bootstrap value of 85% means that the corresponding split occurred in 85% of bootstrap replicate trees under the chosen analysis procedure. It is not an 85% probability that the clade is true. Support depends on the alignment, taxon sampling, model, inference method and number of replicates.

High support indicates consistency under column resampling, but systematic error can be consistently supported. Low support indicates that the data do not robustly distinguish that split; it should not automatically be interpreted as evidence for a particular alternative.

Reporting a tree responsibly State the data and alignment, inference method, substitution model, rooting method, support procedure and number of replicates. Preserve branch-length units and avoid presenting unsupported nodes as established evolutionary events.

Bibliography

- S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. Basic local alignment search tool. *Journal of molecular biology*, 215(3):403–410, 1990.
- M. Dayhoff, R. Schwartz, and B. Orcutt. A model of evolutionary change in proteins. *Atlas of protein sequence and structure*, 5:345–352, 1978.
- S. R. Eddy. Where did the BLOSUM62 alignment score matrix come from? *Nature Biotechnology*, 22(8):1035–1036, a.
- S. R. Eddy. What is dynamic programming? *Nature biotechnology*, 22(7):909–910, b.
- R. C. Edgar. Muscle: multiple sequence alignment with high accuracy and high throughput. *Nucleic acids research*, 32(5):1792–1797, 2004.
- S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992.
- S. Karlin and S. F. Altschul. Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6):2264–2268, 1990.
- K. Katoh, K. Misawa, K.-i. Kuma, and T. Miyata. Mafft: a novel method for rapid multiple sequence alignment based on fast Fourier transform. *Nucleic acids research*, 30(14):3059–3066, 2002.
- M. Kozak. An analysis of 5'-noncoding sequences from 699 vertebrate messenger RNAs. *Nucleic Acids Research*, 15(20):8125–8148, 1987.
- K. Lange. *Mathematical and statistical methods for genetic analysis*, volume 488. Springer, 2002.
- T. Lassmann and E. L. Sonnhammer. Kalign—an accurate and fast multiple sequence alignment algorithm. *BMC bioinformatics*, 6:1–9, 2005.
- S. B. Needleman and C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of molecular biology*, 48(3):443–453, 1970.
- C. Notredame, D. G. Higgins, and J. Heringa. T-Coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of molecular biology*, 302(1):205–217, 2000.
- T. D. Schneider and R. M. Stephens. Sequence logos: a new way to display consensus sequences. *Nucleic Acids Research*, 18(20):6097–6100, 1990.
- T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of molecular biology*, 147(1):195–197, 1981.
- Z. Songyang, S. Blechner, N. Hoagland, M. F. Hoekstra, H. Piwnicka-Worms, and L. C. Cantley. Use of an oriented peptide library to determine the optimal substrates of protein kinases. *Current Biology*, 4(11):973–982, 1994.
- A. Tareen and J. B. Kinney. Logomaker: beautiful sequence logos in Python. *Bioinformatics*, 36(7):2272–2274, 2020.